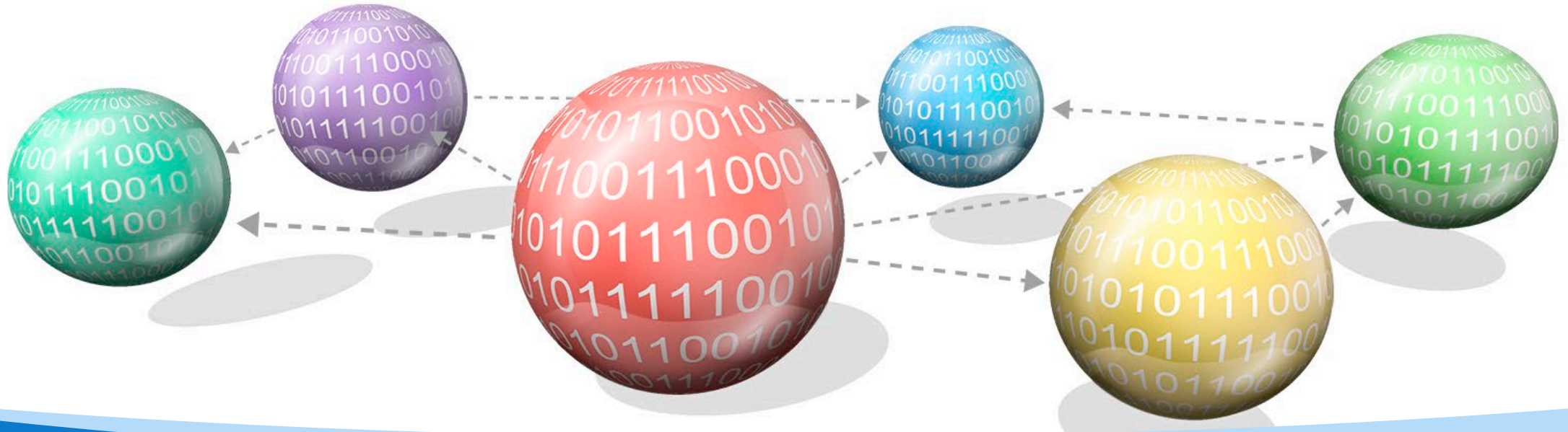




SC2002 Object-Oriented Design & Programming

Chapter 6: UML Model Class Relationships - Class Diagram

Dr. Li Fang

Senior Lecturer, College of Computing and Data Science

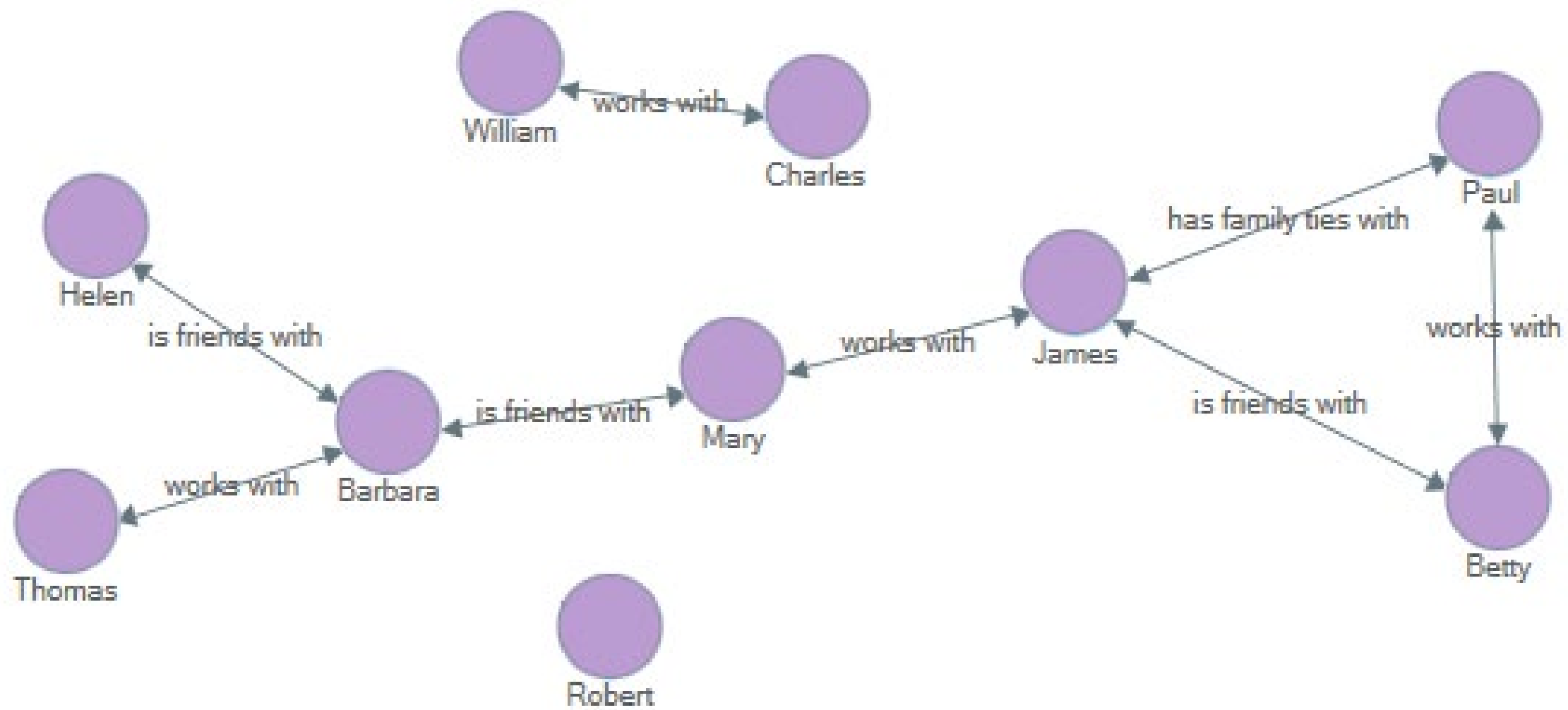


**NANYANG
TECHNOLOGICAL
UNIVERSITY**
SINGAPORE

Why OODP:Real-world Modeling

- OODP is ideal for modeling real-world systems and relationships. Objects can represent tangible things, like users or products, or more abstract concepts, like processes or transactions.
- This modeling makes it intuitive for people to design and think about systems in a way that mirrors real-world scenarios.





Q1



BCE (Boundary–Control–Entity)

- **BCE is an object-oriented modeling pattern** proposed by *Ivar Jacobson* in the **OOSE (Object-Oriented Software Engineering)** method.
- To **structure early-stage system design** — before coding.
- It helps designers assign responsibilities to classes according to their roles in a system's use cases.

BCE (Boundary–Control–Entity)

- **Boundary** → interacts with the outside world (like user interface or API).
- **Control** → manages the **logic or workflow** of each use case.
- **Entity** → holds the **data and business rules** of the system.

BCE separates a system into interface (Boundary), process (Control), and data (Entity) to make programs easier to understand, maintain, and extend.

Boundary (B)

- Represents the **interface between the user and the system**.
- It handles **inputs and outputs**, such as displaying menus, receiving selections, and showing messages.

In the vending machine:

Boundary classes or components include:

Display screen / keypad / buttons – where the user selects drinks.

Coin/Card input – where the user pays.

Dispenser output – where the drink is released.

Example class: VendingMachineUI (reads user input, shows messages).

Control (C)

- Coordinates the **use-case logic** — it tells the system what to do step by step.
- It receives input from the Boundary, checks business rules through Entities, and returns results to the Boundary.

In the vending machine:

Control classes handle the workflow:

Take the selected drink from the UI.

Ask the user to insert coins.

Check if payment is enough.

Command the machine to dispense the drink.

Example class: VendingMachineController (manages the buying process).

Entity (in software design / BCE / UML)

- An **Entity** is a class that represents a **real-world object or concept** that the system needs to remember or manage.
It usually contains **data (attributes)** and sometimes **business rules (methods)** related to that object.

Key Points

- Entities model the **core information** of the system.
- They are often **persistent** — their data may be stored in a file or database.
- They do **not** handle input/output or control flow; they only represent *what something is* and *what it knows*.

An Entity is a blueprint for a real or conceptual thing in the system — it defines the data and rules that describe that thing.

Q1



Entity classes

- They often represent things that exist in the real world (like VendingMachine, Drink, Customer) or important concepts (like Transaction, Invoice).

Feature	BCE (Boundary–Control–Entity)	MVC (Model–View–Controller)
Main Idea	Organize classes by <i>use-case responsibility</i>	Organize code by <i>UI architecture</i>
Boundary / View	<code>Boundary</code> = interface to outside world (UI, API, CLI)	<code>view</code> = user interface display
Control / Controller	<code>control</code> = manages one <i>use-case flow</i> (e.g. "Buy Drink")	<code>controller</code> = handles user input & updates model/view
Entity / Model	<code>Entity</code> = business data & logic (domain objects)	<code>Model</code> = data + business logic

BCE vs MVC

- BCE is considered a **conceptual design model**, or more specifically a:
- **Responsibility assignment model** (who does what).
- **High-level object model** for system analysis.
- It's similar in spirit to MVC (Model–View–Controller), but **BCE is used earlier** in the design process — typically in **use-case modeling**.

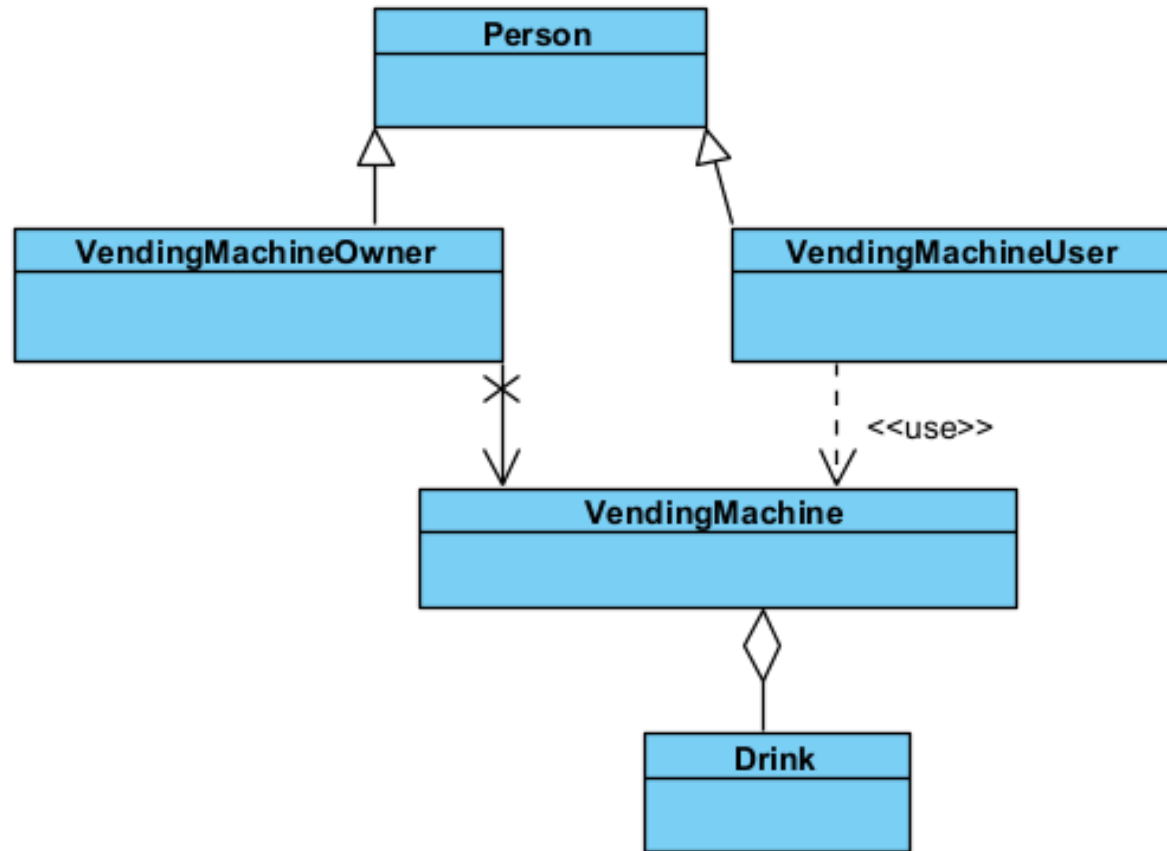
BCE vs Design Patterns

- BCE defines **roles** (responsibility allocation), and Design Patterns provide **implementation solutions** inside those roles.

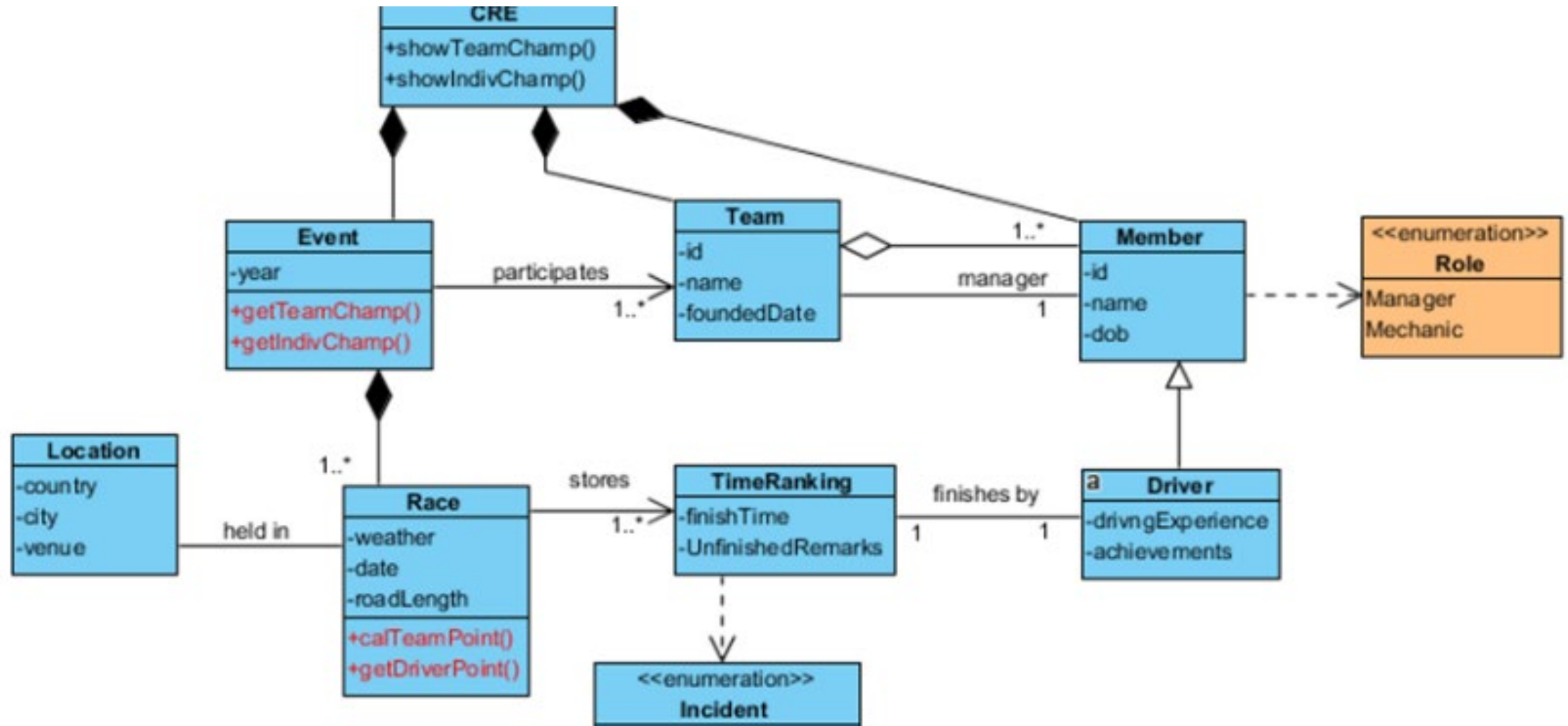
UML Class Diagram

- A UML Class Diagram models the **static structure of a system**:
- It shows:
 - classes
 - attributes
 - methods
- relationships between classes
 - Typical relationships include:
 - association
 - aggregation
 - composition
 - inheritance
 - dependency

Q1



Final product





TOPIC

Topic 1: Relationship

Types of

Class Relationships in Java

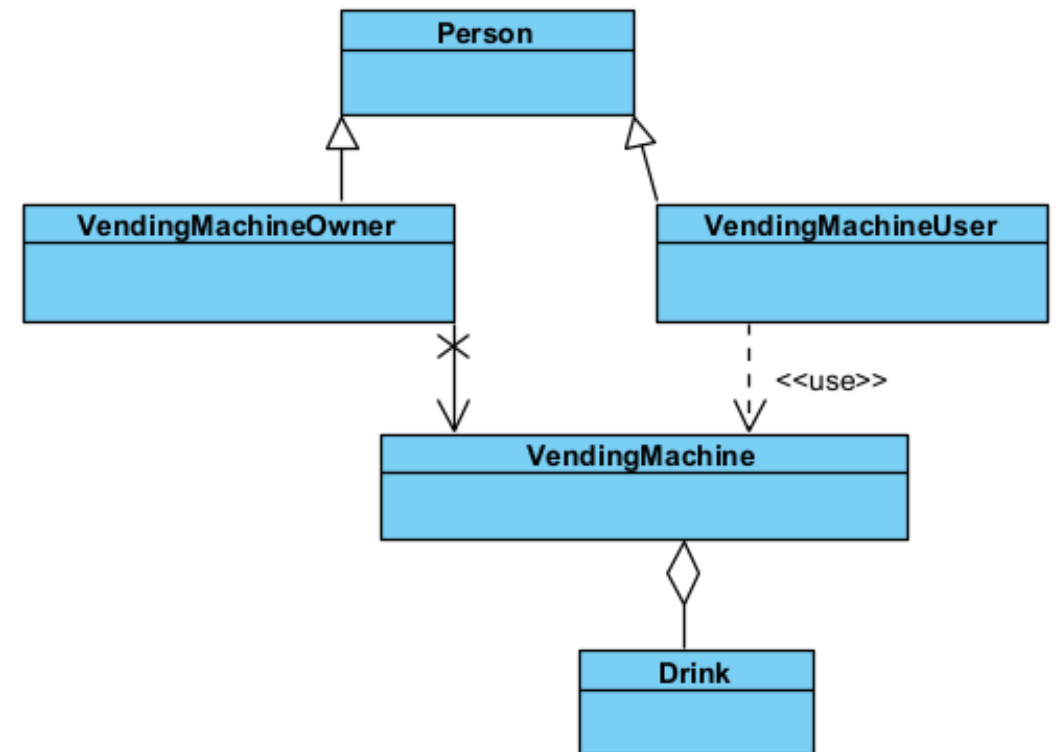
**Uses-A
Relationship**

**Has-A
Relationship**

**Is-A
Relationship**

Aggregation

Composition



Inheritance Hierarchy

Superclass

(parent class or base class)

Super, base, parent, generalised

Shape

- xPos: double
- yPos: double

+ Shape()
+ Shape(xCoor: double, yCoor: double)
+ getXPos(): double
+ getYPos(): double
+ print(): void

Common attributes and methods

is-a

is-a

Rectangle

- width: double
- height: double

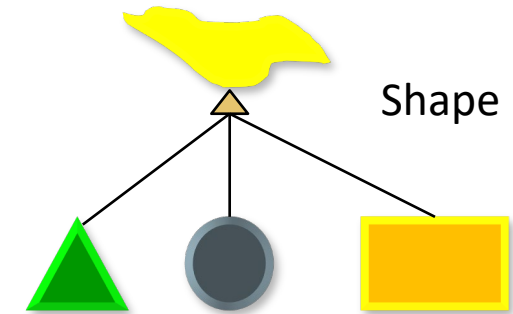
+ Rectangle()
+ Rectangle(xCoor: double, yCoor: double, w: double, h: double)
+ getWidth(): double
+ getHeight(): double
+ findArea(): double
+ findPerimeter(): double
+ print(): void

Additional attributes and methods

Circle

- radius: double

+ Circle()
+ Circle(xCoor: double, yCoor: double, rad: double)
+ getRadius(): double
+ findArea(): double
+ findPerimeter(): double
+ print(): void



Subclasses: Inherits all the instance variables and methods of the superclass.

(EXCEPT those declared as private - accessibility)

Sub, derived, child, specialised

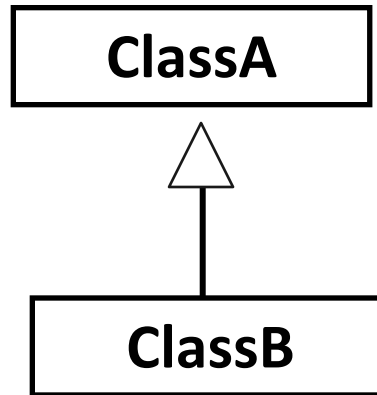


TOPIC

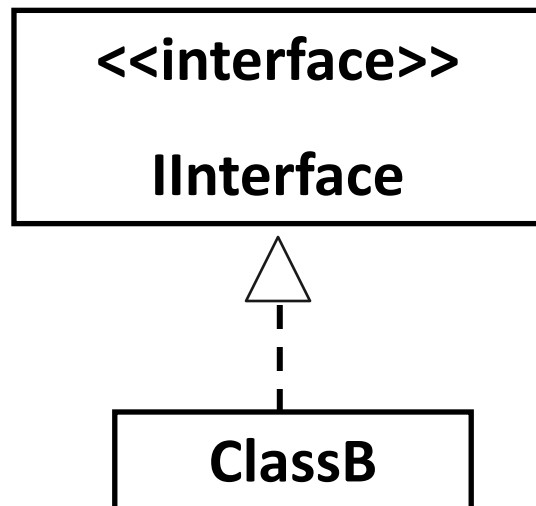
Topic 1.2: Inheritance

UML Class Diagram to Java Code

Generalisation (Inheritance)



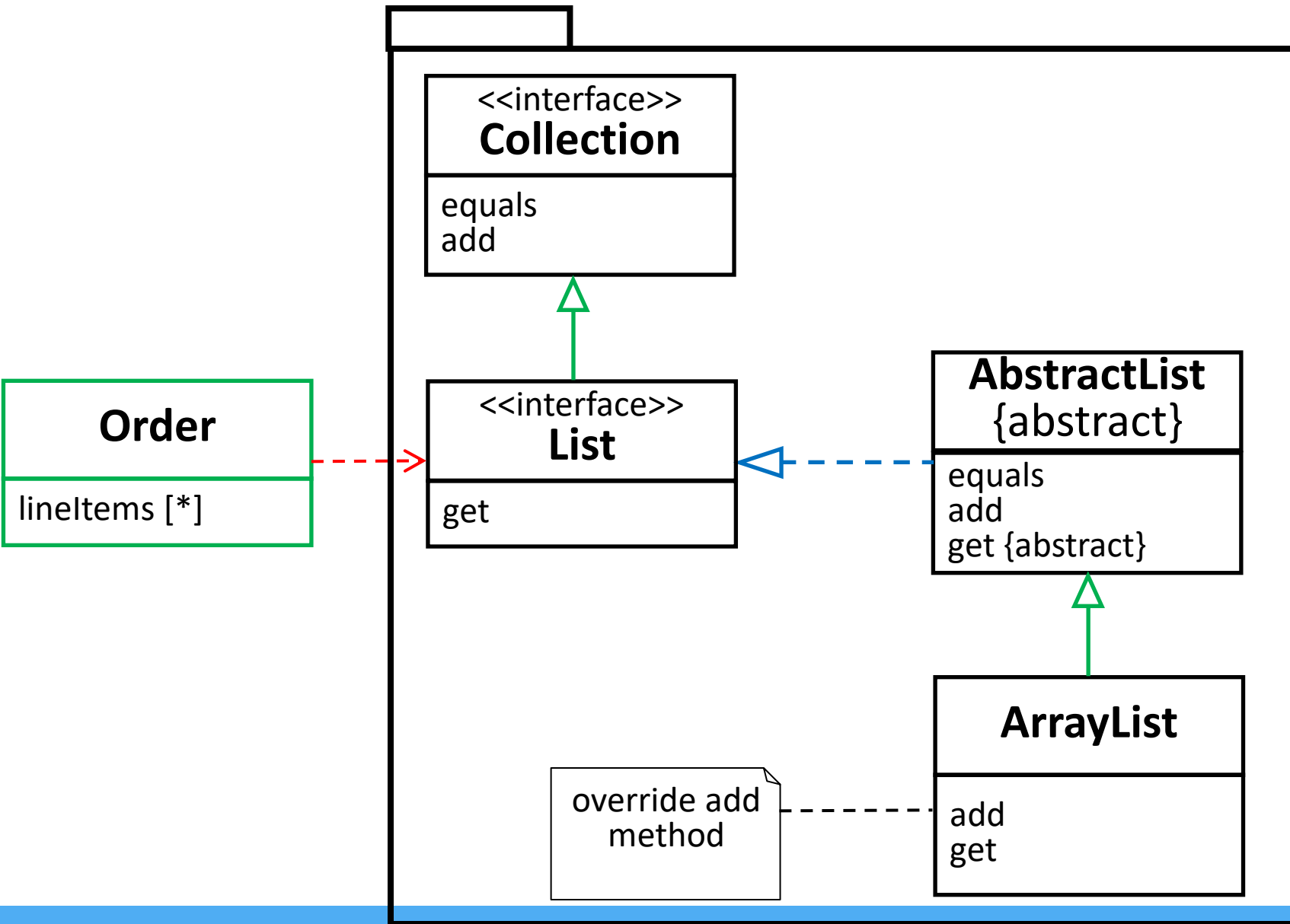
```
class ClassB extends ClassA {  
.....  
}
```



```
class ClassB implements Interface{  
.....  
}
```

Realisation

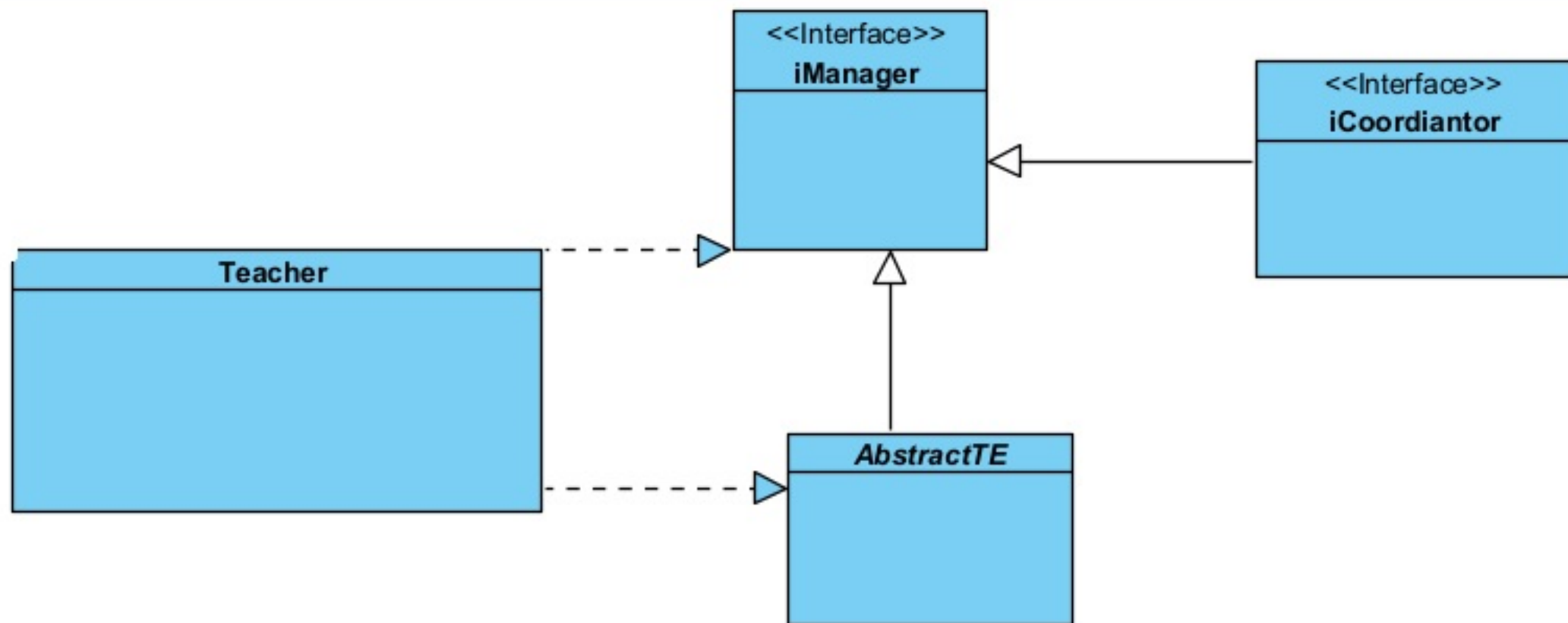
Java Collections Framework



- Interface inherits Interface,
- implementation only happen when class inherits interface.
- Abstract class are classes.



What are the errors in the class diagram?





TOPIC

Topic 1.3: Dependency

Types of

Class Relationships in Java

**Uses-A
Relationship**

**Has-A
Relationship**

**Is-A
Relationship**

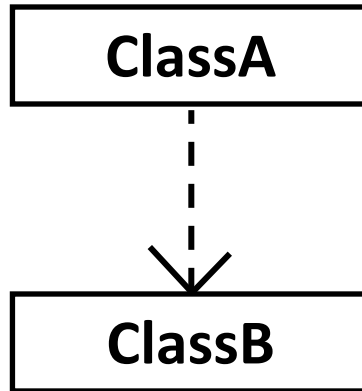
Aggregation

Composition



UML Class Diagram to Java Code

Dependency



They (Dependency Relationships) are not implemented with member variables at all.

```
class ClassA {
    .....
    ClassB doSomething(...) { // as return type
    .....
    ....
}
```

```
class ClassA {
    .....
    <ReturnType> doSomething(ClassB b, ...) { // as method argument
    .....
    ....
}
```

```
class ClassA {
    .....
    <ReturnType> doSomething(.....) {
        ClassB b = new ClassB() ; // as variable inside method
        ....
    }
}
```

```
class Person {
    private String name;
    private double money;

    public Person(String name, double money) {
        this.name = name;
        this.money = money;
    }

    // The Person depends on VendingMachine to purchase a Drink
    public void purchaseDrink(VendingMachine machine, String drinkName) {
        Drink drink = machine.dispense(drinkName, money);
        if (drink != null) {
            money -= drink.getPrice();
            System.out.println(name + " purchased a " + drink.getName() + " for $" + drink.getPrice()
                               + ". Remaining balance: $" + money);
        } else {
            System.out.println(name + " could not purchase " + drinkName + ".");
        }
    }
}
```

```
public class Main {
    public static void main(String[] args) {
        // Create a vending machine with some drinks
        VendingMachine machine = new VendingMachine();

        // Create a Person with a certain amount of money
        Person alice = new Person("Alice", 2.00);

        // Alice attempts to purchase a drink from the vending machine
        alice.purchaseDrink(machine, "Coke");
    }
}
```



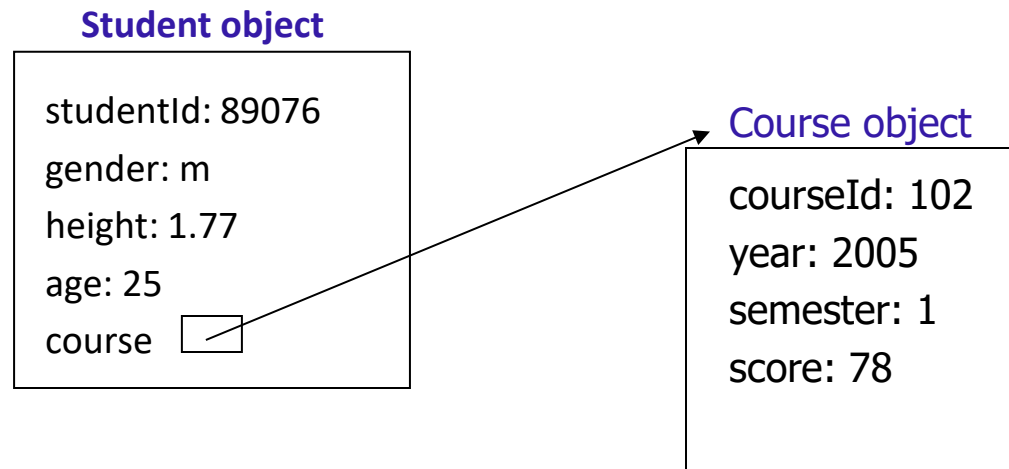
TOPIC

Topic 1.1: Association

Object Composition

- An object can include other objects as its **data** member. This is called **object composition**.
- A class contains **object references** from other classes as its **instance variables**.
- Object references are of reference data types.
- **In OO, this is called the “has-a” relationship.**
- Example: a student *has a* course.

```
public class Student {  
    private int    studentId;  
    private char   gender;  
    private double height;  
    private int    age;  
    private Course course;
```



```
public class Course {  
    private int courseId;  
    private int year;  
    private int semester;  
    private int score;
```

```
public class Student {  
    private int    studentId;  
    private char   gender;  
    private double height;  
    private int    age;  
    private Course course;}  
}
```

A student and a course
A sports team and its players
A house and its rooms



Based on the program below, there are four statements,
Which of the following statement(s) is/are correct?

Based on the program below, there are four statements, Which of the following statement(s) is/are correct?

```
public class A {  
    private C c;  
    public void myMethod(B b) {  
        b.callMethod();  
    }  
}
```

- I. Relationship between A and B is Association
- II. Relationship between A and C is Association
- III. Relationship between A and B is Dependency
- IV. Relationship between A and C is Dependency

- A. I and II
- B. I and IV
-  C. II and III
- D. III and IV

Dependency vs Association

```
class Teacher {  
    String name;  
    void teach(Student student) {  
        System.out.println(name + " is teaching " + student.name);  
    }  
}
```

```
class Student {  
    String name;  
}
```

```
public class Test {  
    public static void main(String[] args)  
    {  
        Teacher teacher = new Teacher();  
        teacher.name = "Mr. Smith";  
  
        Student student = new Student();  
        student.name = "John";  
  
        teacher.teach(student);  
    }  
}
```

- Dependency means that one class temporarily uses another class, usually through a method or parameter.
- The classes are not linked for the long term.
- In the demo code, the Teacher class depends on Student to perform its teach method, but there's no persistent relationship between the two. The Student object is passed as an argument, meaning the Teacher depends on Student only while calling the teach method.
- In dependency, the relationship is limited to method parameters (e.g., passing a Student object to a teach method), and once the method execution is over, the relationship ends. The teacher cannot "remember" which students were taught, nor can they modify the state of the students later on.

Association

```
import java.util.List;
import java.util.ArrayList;

class Teacher {
    String name;
    List<Student> students; // Association with Student (Teacher knows about students)

    Teacher() {
        students = new ArrayList<>(); // Initialize the list of students
    }

    void addStudent(Student student) {
        students.add(student); // Add a student to the teacher's list
    }

    void teach() {
        for (Student student : students) {
            System.out.println(name + " is teaching " + student.name);
        }
    }
}
```

```
class Student {
    String name;
}
```



```
public class Test{  
    public static void main(String[] args) {  
        Teacher teacher = new Teacher();  
        teacher.name = "Mr. Smith";  
  
        Student student1 = new Student();  
        student1.name = "John";  
  
        Student student2 = new Student();  
        student2.name = "Jane";  
  
        // Establish association by adding students to the teacher's list  
        teacher.addStudent(student1);  
        teacher.addStudent(student2);  
  
        // Teacher teaches the students (association)  
        teacher.teach();  
    }  
}
```

Key Takeaway:

- Dependency is short-term and based on method calls.
- Association is long-term and allows richer interactions and modifications over the lifecycle of the objects involved.

The background of the slide features a dense, out-of-focus arrangement of 3D block letters in various shades of gray. The letters are scattered across the upper half of the image, creating a textured, word-like effect. In the center of this background, the word "TOPIC" is prominently displayed in a bright red, 3D, sans-serif font. The letters of "TOPIC" are slightly larger and more defined than the background letters, making them stand out. Below the letter-filled area, there is a solid blue horizontal band that serves as a background for the title text.

TOPIC

Topic 2: More on Association (uni-directional vs bi-directional)

Association

```
import java.util.List;
import java.util.ArrayList;

class Teacher {
    String name;
    List<Student> students; // Association with Student (Teacher knows about students)

    Teacher() {
        students = new ArrayList<>(); // Initialize the list of students
    }

    void addStudent(Student student) {
        students.add(student); // Add a student to the teacher's list
    }

    void teach() {
        for (Student student : students) {
            System.out.println(name + " is teaching " + student.name);
        }
    }
}
```

```
class Student {
    String name;
}
```

```
public class Test{
    public static void main(String[] args) {
        Teacher teacher = new Teacher();
        teacher.name = "Mr. Smith";

        Student student1 = new Student();
        student1.name = "John";

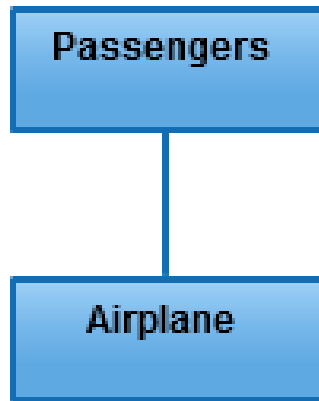
        Student student2 = new Student();
        student2.name = "Jane";

        // Establish association by adding students to the teacher's list
        teacher.addStudent(student1);
        teacher.addStudent(student2);

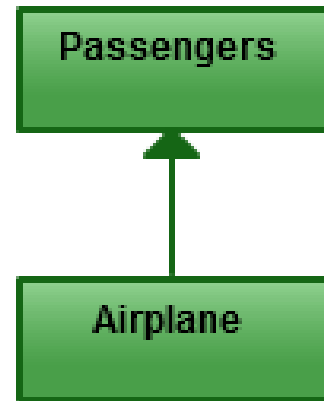
        // Teacher teaches the students (association)
        teacher.teach();
    }
}
```



Which statement is correct regarding the relationship between Teacher and Student in previous program?



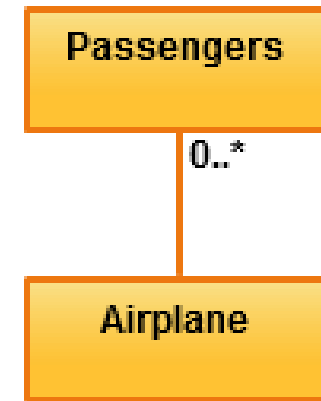
Association



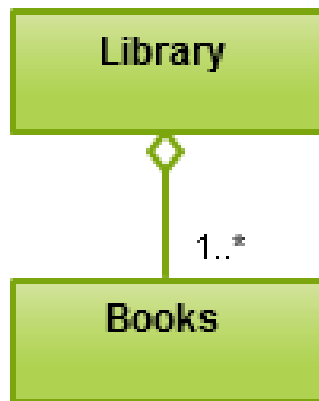
**Directed
Asscoation**



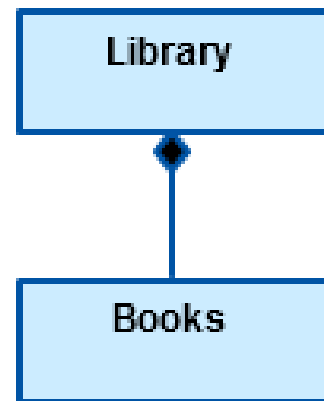
**Reflexive
Assciation**



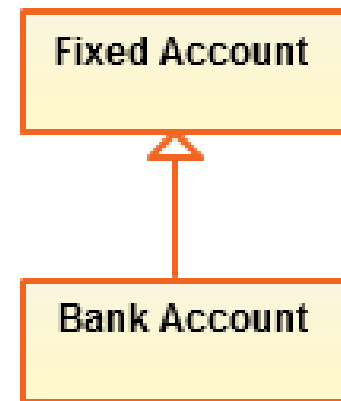
Multiplicity



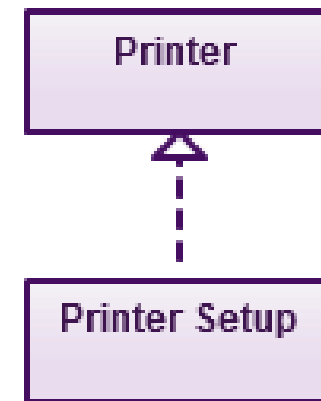
Aggregation



Composition



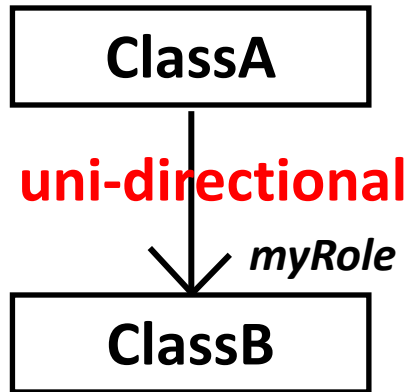
Inheritance



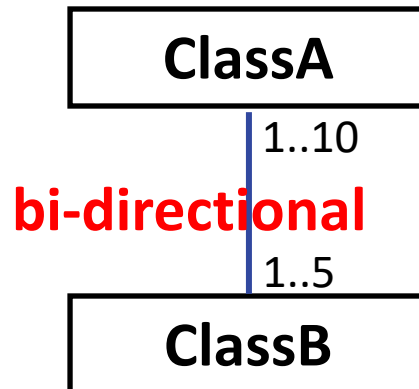
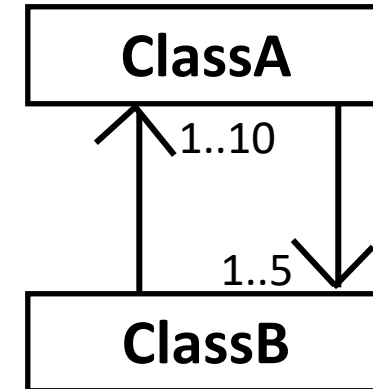
Realization

Association in Java Code

Association



```
class ClassA {  
    ClassB myRole ; // attribute  
    .....  
    ....  
}
```



```
class ClassA {  
    ClassB[ ] bObjs = new ClassB[ 5 ]; // attribute  
    .....  
    ..... bObjs[0] = new ClassB(this) ;  
    .....  
}
```

```
class ClassB {  
    ClassA[ ] aObjs = new ClassA[ 10 ]; // attribute  
    .....  
    ..... public ClassB(ClassA a) { aObjs[0] = a ; ...}  
    .....  
}
```

Object
composition



Which of the following statement(s) describe(s) the relationship between a student and a teacher correctly?

Student

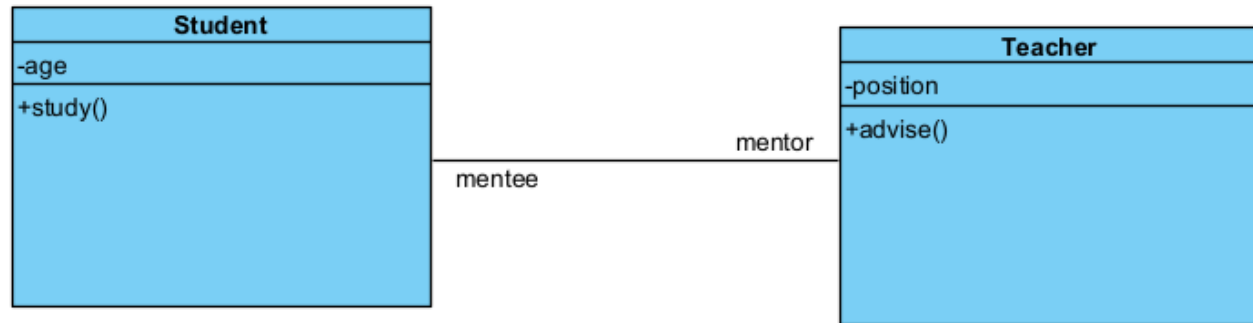
Teacher

A student knows the information of his teachers and A teacher knows the information of his students

A student knows the information of his teachers but a teacher does not need to know the information of his students

A teacher knows the information of his students but a student does not need to know the information of his teachers

Association



- Association between classes is bi-directional by default.
- A straight line

```
public class Student {

    Teacher mentor;
    private int age;

    public void study() {
        // TODO - implement Student.study
        throw new
        UnsupportedOperationException();
    }

}
```

```
public class Teacher {

    Student mentee;
    private int position;

    public void advise() {
        // TODO - implement Teacher.advise
        throw new
        UnsupportedOperationException();
    }

}
```

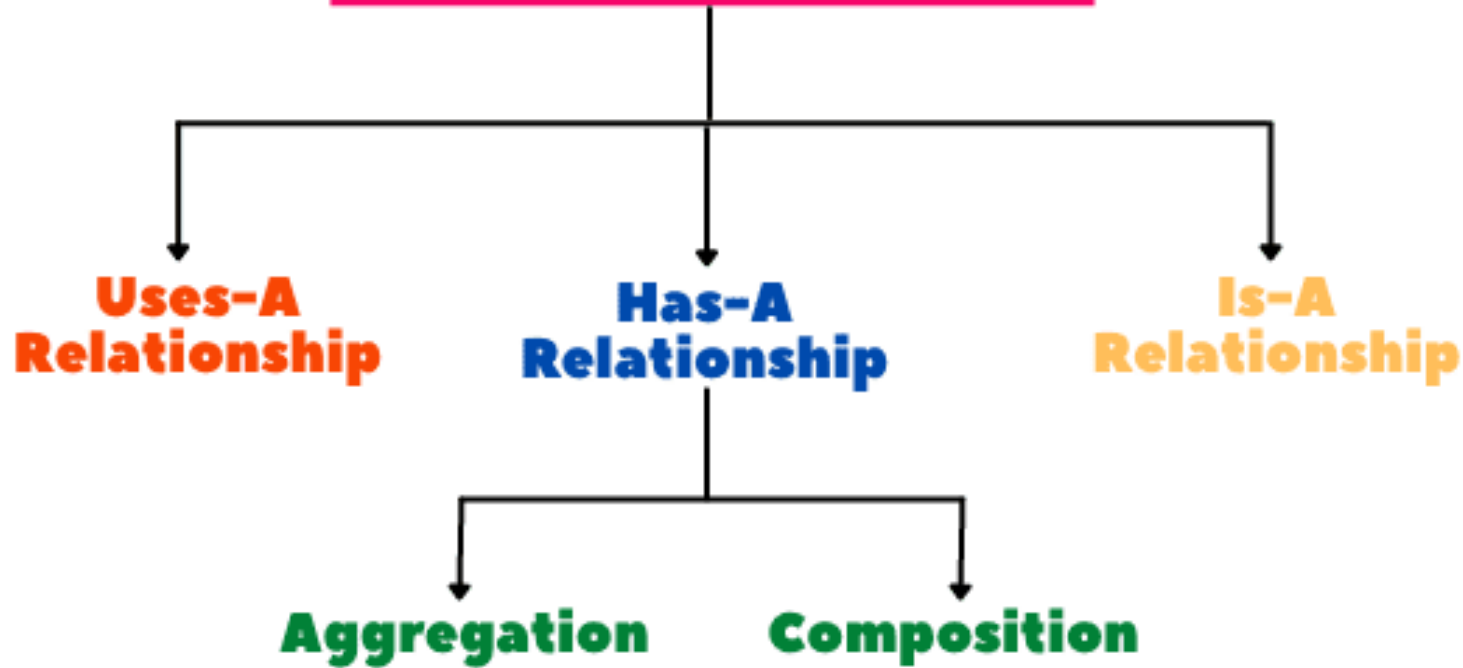


TOPIC

Topic 3: Association : part of

Types of

Class Relationships in Java



Is part of

A has a B

B is part of A

Has-a

- **Association:** A generic relationship where one class uses or interacts with another. It represents a "has-a" relationship without implying strong ownership or lifecycle control. (e.g., A teacher teaches students.)
- **Aggregation:** A special type of association that represents a "whole-part" relationship but with weaker ownership. The lifecycle of the part is independent of the whole. (e.g., A library contains books, but books can exist without the library.)
- **Composition:** A stronger form of aggregation where the "whole" has full ownership of its parts, and the parts cannot exist independently. (e.g., A house is composed of rooms, and the rooms do not exist independently of the house.)

Student

TutorialGroup





Which of the following statement(s) describe(s) the relationship between student and tutorial group correctly?

Which of the following statement describes the relationship between student and tutorial group correctly?

- A. A student is a special tutorial group
- B. A student uses a tutorial group
-  C. A tutorial group has a student/students
-  D. A student is part of a tutorial group
- E. All of above
- F. None of above

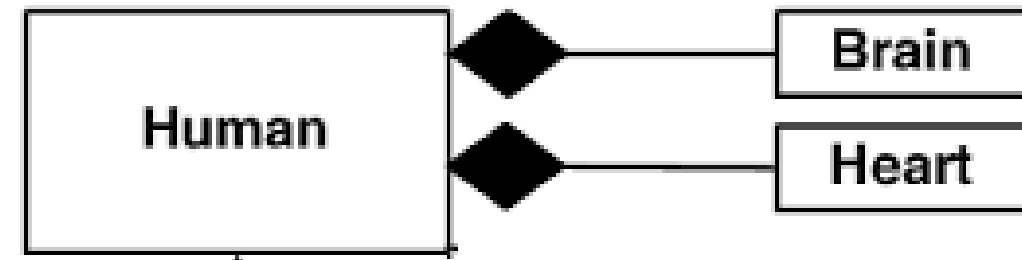
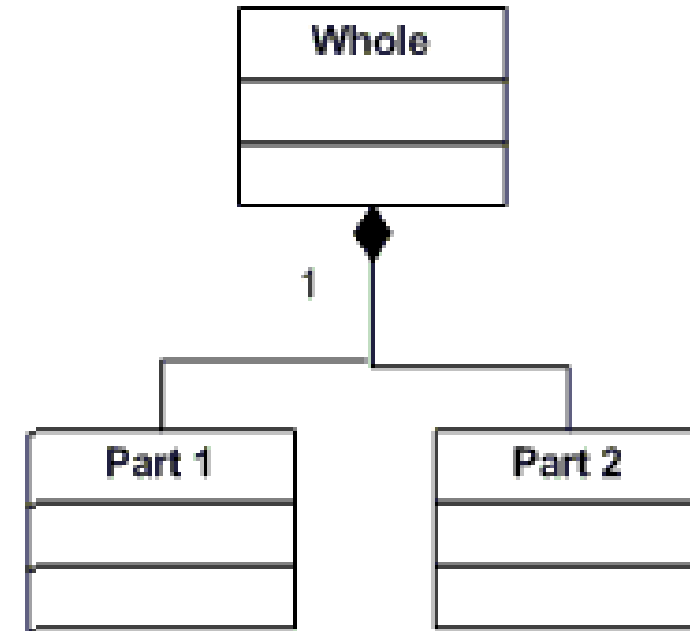
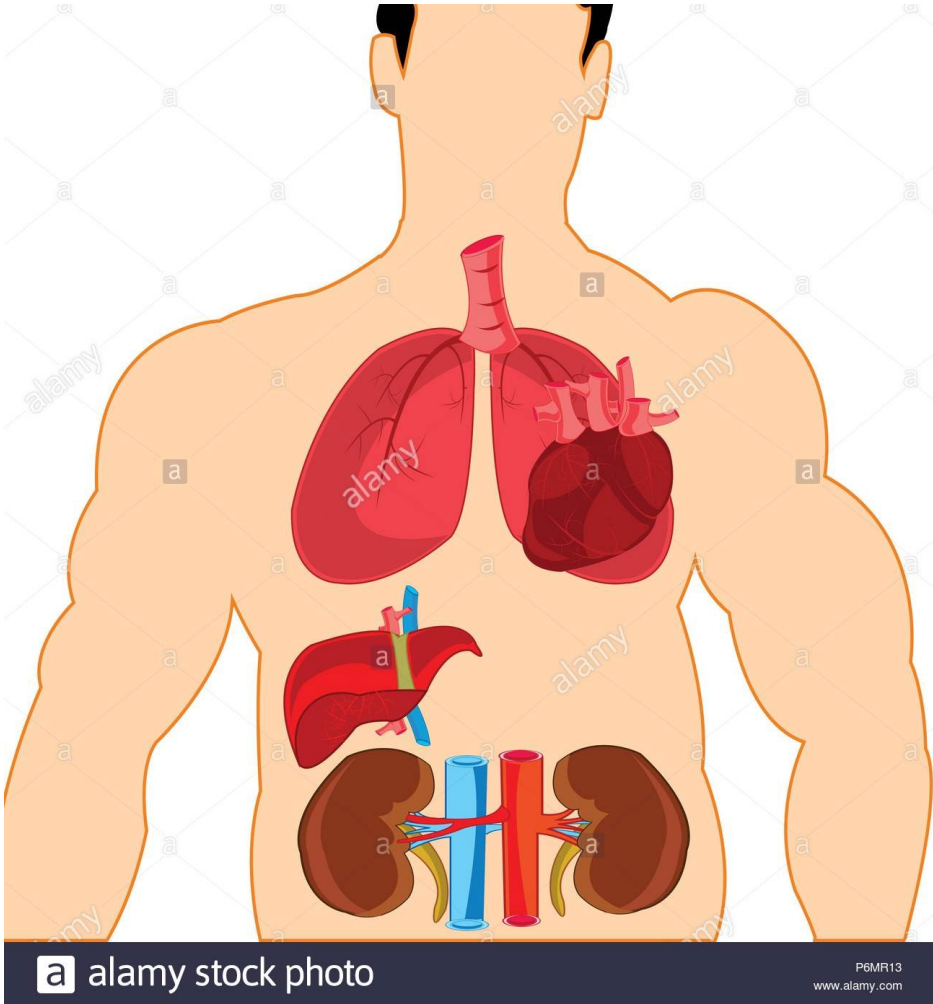
slido

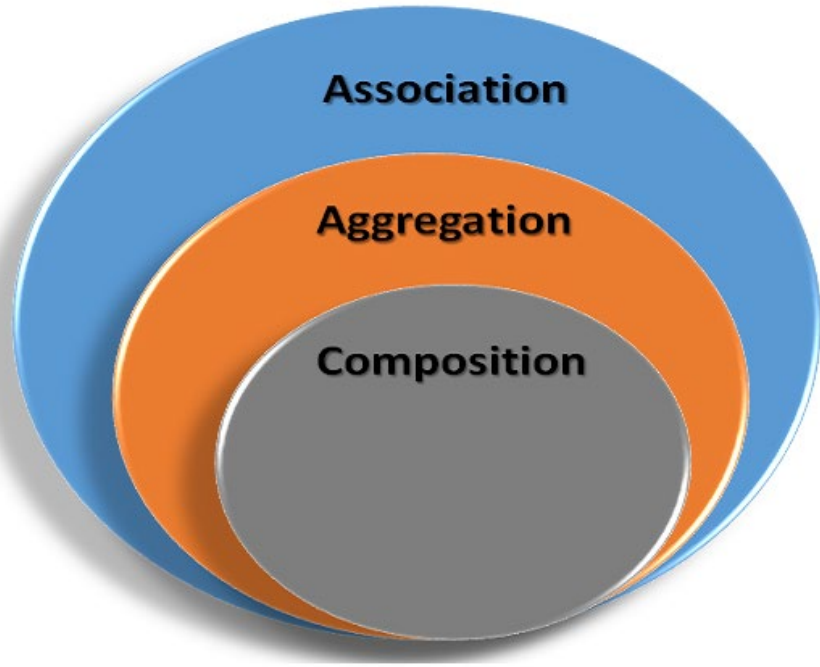


In UML (Unified Modeling Language), how would you best describe the relationship between a Person and their Hands?

① Start presenting to display the poll results on this slide.

Best example of composition (No transplant)





- Association is a relationship between two objects.
- Aggregation is a special form of association.
- Composition is a special form of aggregation.

Association (general relationship)

└─ Aggregation (shared ownership)

└─ Composition (strong ownership)

1. Association (generic relationship)

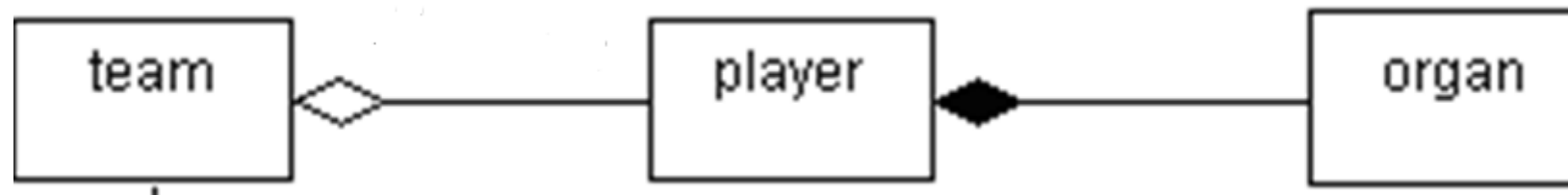
A general relationship between classes where objects know about or interact with each other.

2. Aggregation (weak whole–part)

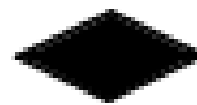
A special association where one object **contains** others, but the parts **can exist independently**.

3. Composition (strong whole–part)

A stronger form where the **part cannot exist without the whole**.



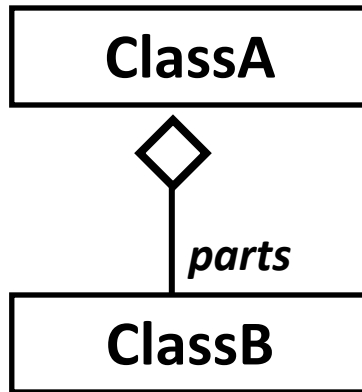
Aggregation



Composition

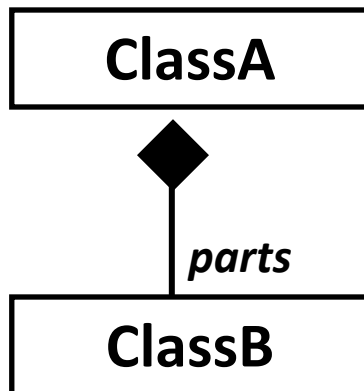
UML Class Diagram to Java Code

Aggregation



```
class ClassA {
    Vector parts = new Vector(); // attribute
    ..... // parts stores a collection of ClassB objs
    public void addClassB(ClassB b) {
        .....// added through reference
        parts.add(b);
    }
    .....
} // use of Vector class is just to show dynamic (*) Collection,
// can be ArrayList, etc OR array[ ] (if size is fixed)
```

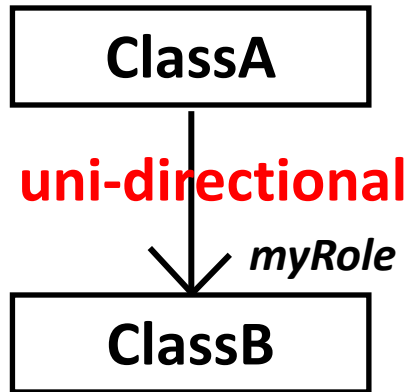
Composition



```
class ClassA {
    Vector parts = new Vector(); // attribute
    .....
    public ClassA( ) {
        ClassB b1 = new ClassB(); // created in class
        ClassB b2 = new ClassB();
        parts.add(b1);
        parts.add(b2);
    }
    .....
}
```

Association in Java Code

Association



```
class ClassA {  
    ClassB myRole ; // attribute  
    .....  
    ....  
}
```


Aggregation denotes whole/part relationships whereas associations do not. However, there is not likely to be much difference in the way that the two relationships are implemented. That is, it would be very difficult to look at the code and determine whether a particular relationship ought to be aggregation or association.

- Robert C. Martin, UML Tutorial: Part 1 — Class Diagrams

slido



Which of the following statement(s) describe(s) the relationship between a student and his umbrella correctly?

① Start presenting to display the poll results on this slide.

Student

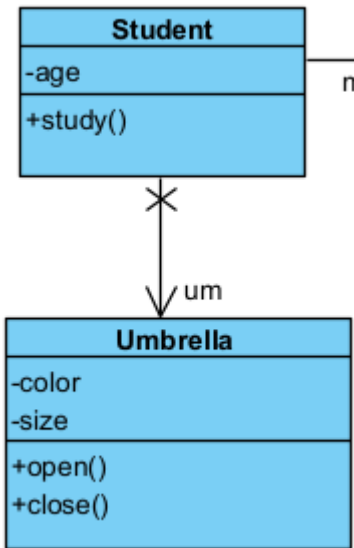
Umbrella

A student owns an umbrella

A student knows the information of his umbrella

An umbrella owns a student

An umbrella knows the information of his owner, who is a student



```

public class Student {

    Teacher mentor;
    private int age;
    Umbrella um;

    public void study() {
        // TODO - implement
        Student.study
        throw new
        UnsupportedOperationException();
    }

}
  
```

```

public class Umbrella {

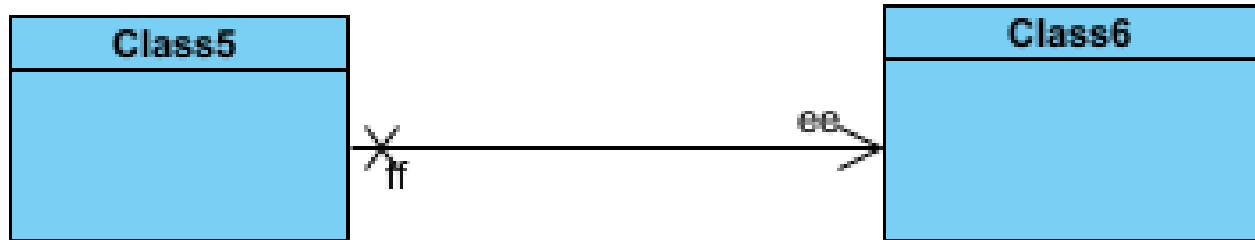
    private int color;
    private int size;

    public void open() {
        // TODO - implement
        Umbrella.open
        throw new
        UnsupportedOperationException();
    }

    public void close() {
        // TODO - implement
        Umbrella.close
        throw new
        UnsupportedOperationException();
    }

}
  
```

Visual paradigm can only recognize the uni-directional association in the following way. Explicitly close one end by setting Navigable to be false.



Auto generated
code:

```
public class Class5 {  
  
    Class6 ee;  
  
}
```

```
public class Class6 {  
  
}
```

Association Specification

General

Name:

Visibility:

Association End From

Role:

Element:

Multiplicity:

Navigable:

Association End To

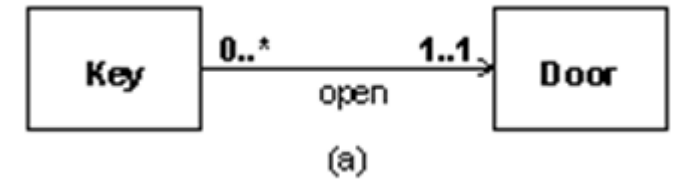
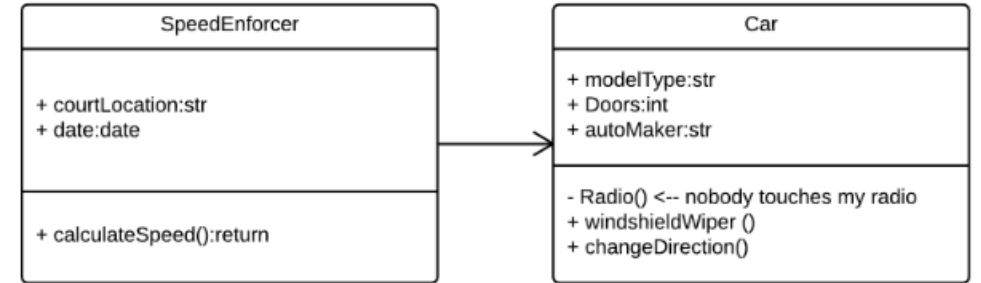
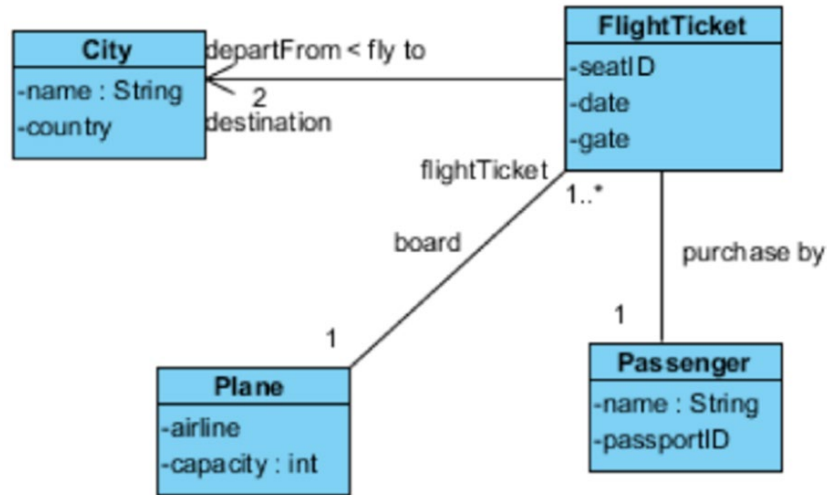
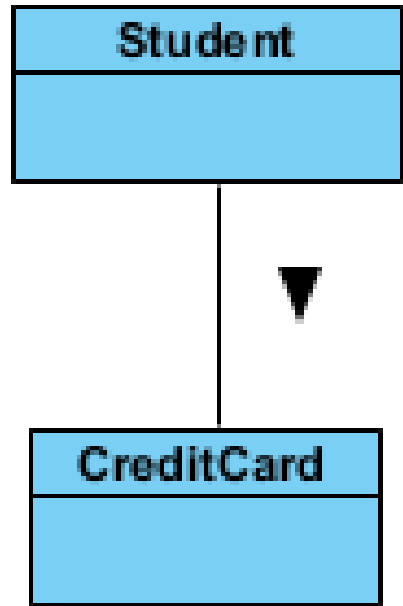
Role:

Element:

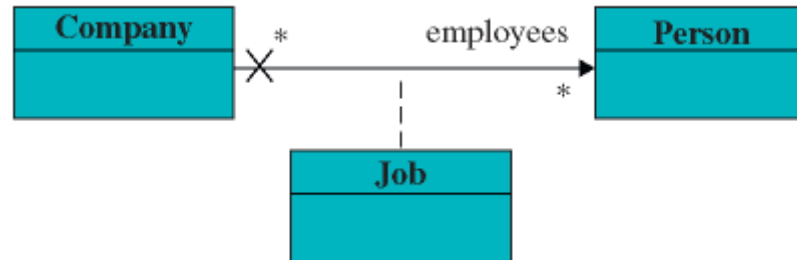
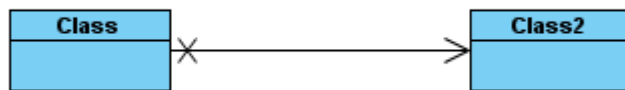
Multiplicity:

Navigable:

Directed Association (Uni-directional Association) in different UML tools (all are accepted in assignment class diagram drawing)



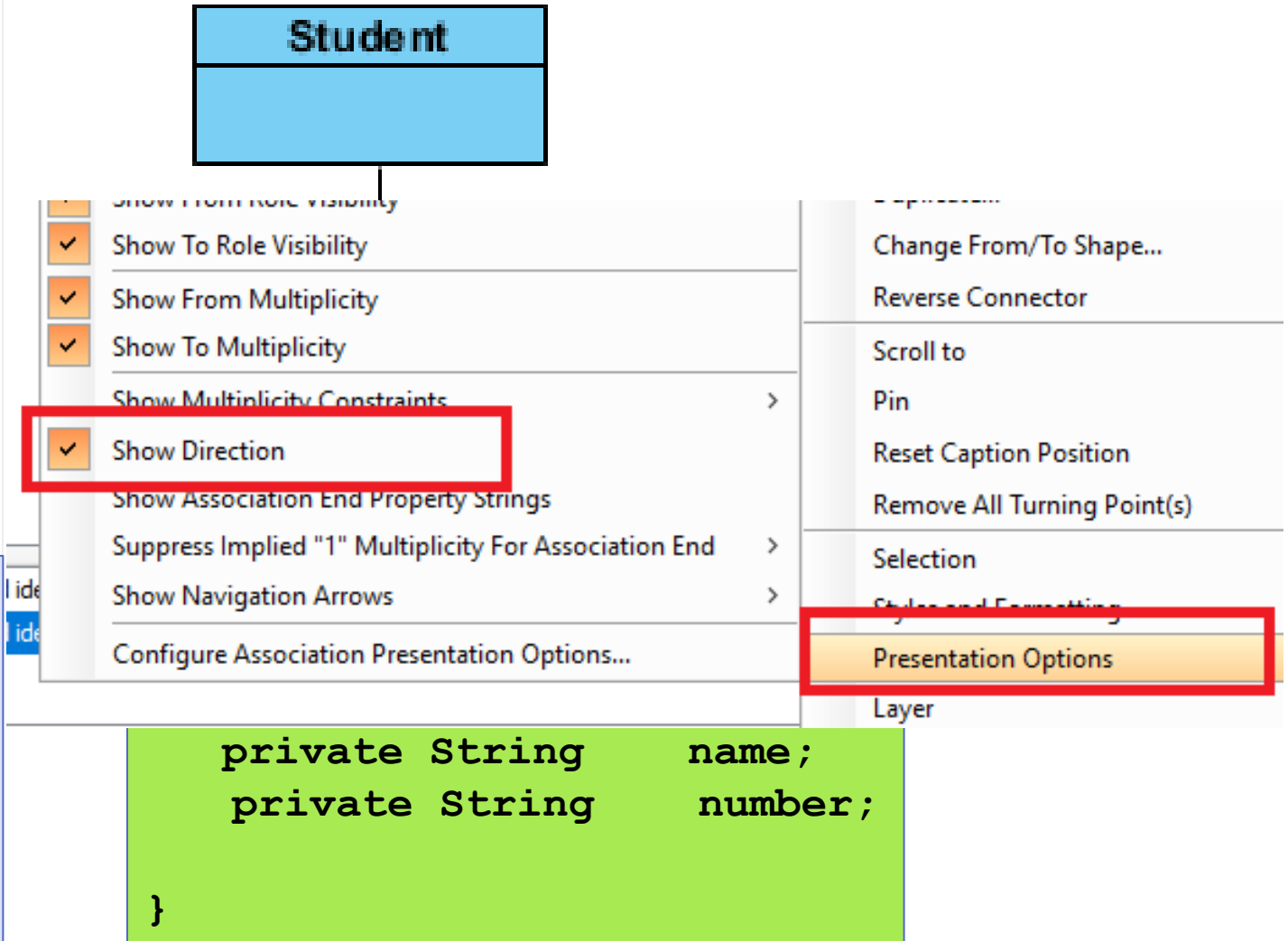
unidirectional



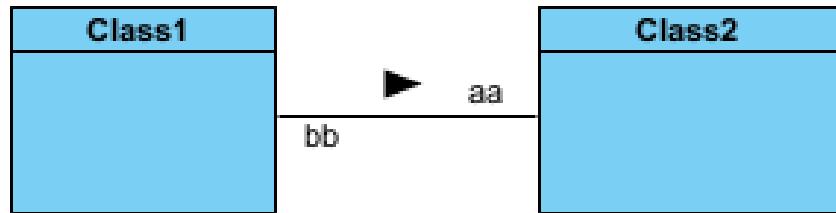
Directed Association (Uni-directional Association)

You can define the flow of the association by using a directed association. The arrowhead identifies the container-contained relationship.

```
public class Student {  
    private int    studentId;  
    private char   gender;  
    private double height;  
    private int    age;  
    private CreditCard[] cards;  
    ...  
}
```



Human being knows clearly that the following two indicate uni-directional associations. However, Visual paradigm code auto generator cannot recognize the following two uni-directional associations, treat as bi-directional association in auto coding.



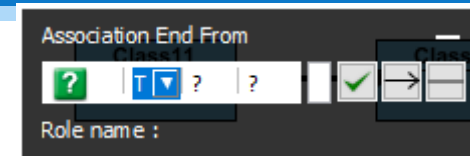
Auto generated
code:

```

public class Class1 {
    Class2 aa;
}
  
```

```

public class Class2 {
    Class1 bb;
}
  
```



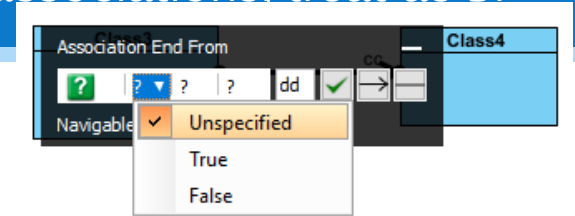
Auto generated
code:

```

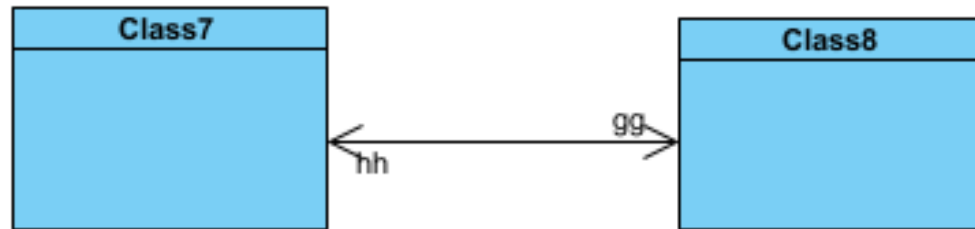
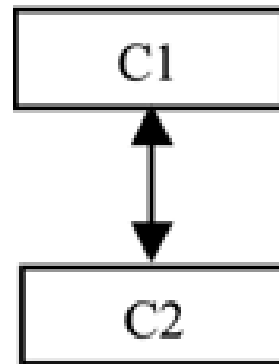
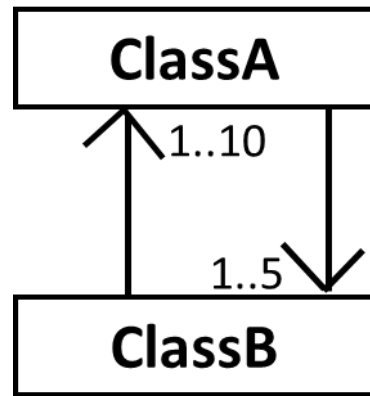
public class Class3 {
    Class4 cc;
}
  
```

```

public class Class4 {
    Class3 dd;
}
  
```



bi-directional Association in different UML tools (all are accepted in exam class diagram drawing)



The default bi-directional association is a straight line without arrows. How to draw bi-directional association with explicit arrows in Visual paradigm?

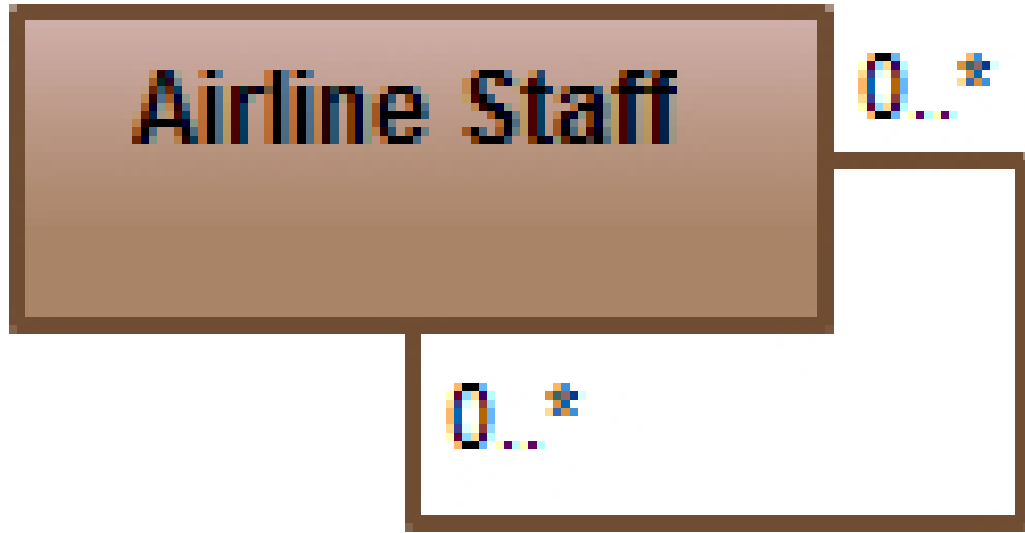


The screenshot displays the Visual Paradigm software interface. On the left, a UML class diagram shows a bi-directional association between Class7 and Class8. The association line has a small circle at the Class7 end and an arrowhead at the Class8 end. The association is labeled with 'hh' near Class7 and '99' near Class8. On the right, the 'Role B (Class8)' context menu is open. The menu items are as follows:

- Role B (Class8)
 - Navigable >
 - Multiplicity >
 - Visibility >
 - Aggregation Kind >
 - Edit Role Name...
 - Qualifier...
 - Owned by >
 - Sub Diagrams >
- Edit
 - Cut
 - Copy >
 - Delete >
 - Duplicate...
 - Change From/To Shape...
 - Reverse Connector
 - Scroll to Class7
 - Pin >
 - Reset Caption Position
 - Remove All Turning Point(s)
- Selection >

At the bottom of the context menu, there are three options: 'Follow Diagram (Hide Arrows With Two-Way Navigability)', 'Show All Arrows and Crosses' (highlighted with a red rectangle), and 'Hide All Arrows and Crosses'. To the right of the context menu, the 'Show Caption' and 'Paint Through Label' sections are visible, with 'Follow Diagram (Yes)' and 'Follow Diagram (No)' options respectively. The 'Show Navigation Arrows' option is also visible and highlighted with a red rectangle.

Reflexive Association



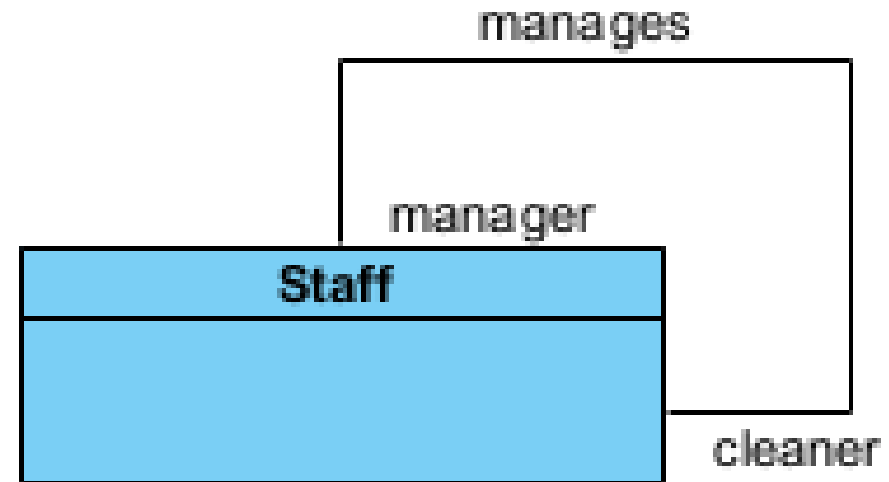
No separate symbol.
However, the
relation will point
back at the same
class.

This occurs when a class may have multiple functions or responsibilities. For example, a staff member working in an airport may be a pilot, aviation engineer, a ticket dispatcher, a guard, or a maintenance crew member. If the maintenance crew member is managed by the aviation engineer there could be a managed by relationship in two instances of the same class.

Reflexive Association

```
public class Staff{  
    Staff manager;  
    Staff cleaner;  
}
```

A manager, who
is a staff,
manages a
cleaner, who is
a staff as well.



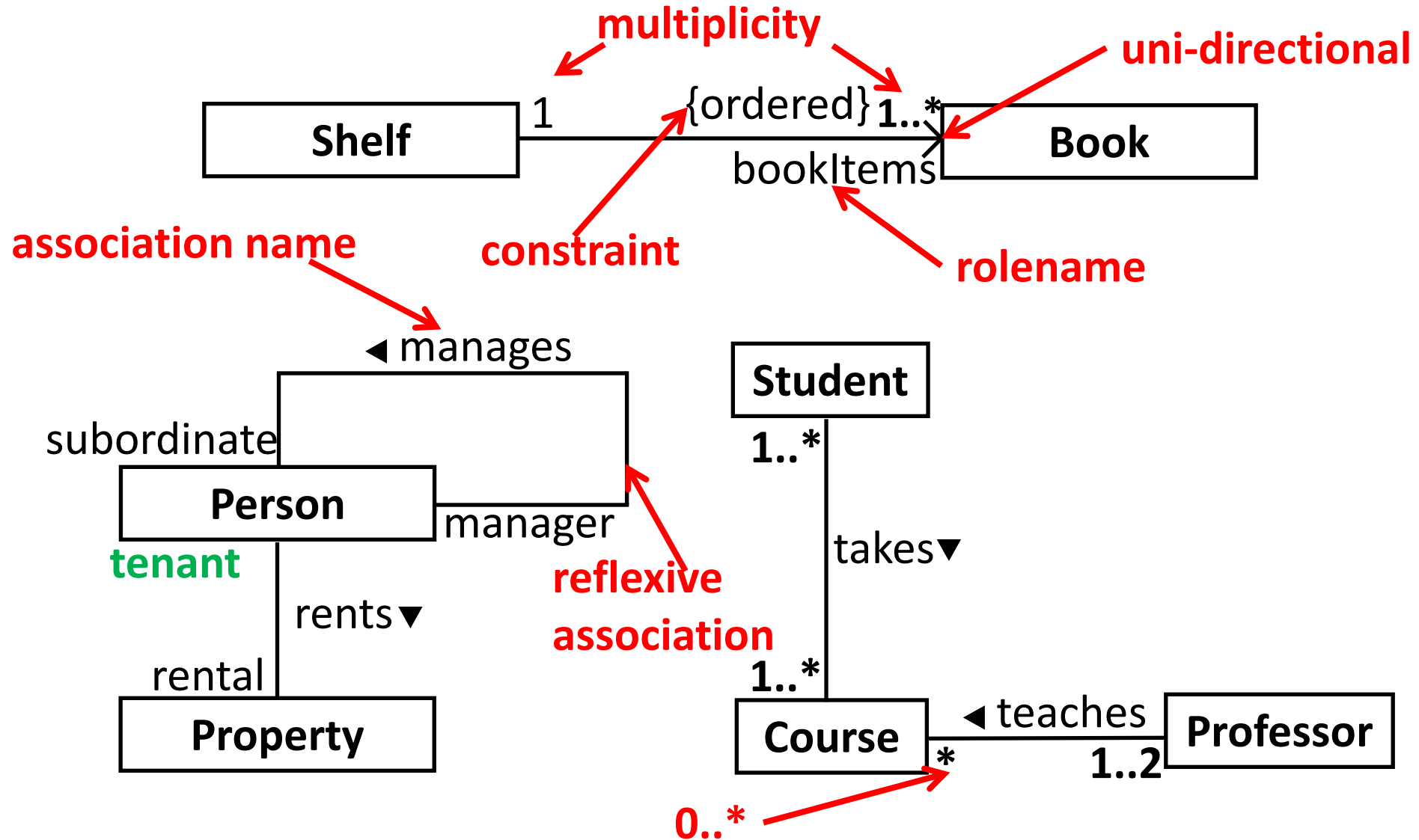
Split to subclasses?

- The answer really depends on your design requirements and the nature of the relationship:
- **When to Keep a Reflexive Association:**
If the relationship simply represents a hierarchy or recursive link (for example, an employee managing another employee) without any significant differences in behavior or attributes, a reflexive association is usually more straightforward and easier to maintain.
- **When to Use Subclasses:**
If the roles or behaviors within the relationship differ substantially—meaning each “side” of the association has unique properties or responsibilities—it might be beneficial to break the entity into subclasses. This can help encapsulate the specific behavior of each role and improve clarity, albeit at the cost of added complexity in the class hierarchy.

Tutorial 6 Q1

- Given the following set of classes, draw a Class Diagram to show the appropriate **relationship** between them. Add **multiplicity, role name and association name**, if necessary:
 - Library, LibraryResource, Book, AudioVisual, Magazine
 - Driver, Car, Wheel, Engine
 - Plane, City, Passenger, FlightTicket
 - Company, Person, Department, Job
 - Product, Inventory, ItemStock, Catalog, Order, OrderLineItem, Manufacturer

Class Diagram - Association





TOPIC

Topic 3: Association name

Which of the following verb is appropriate to link the following two classes?

Student

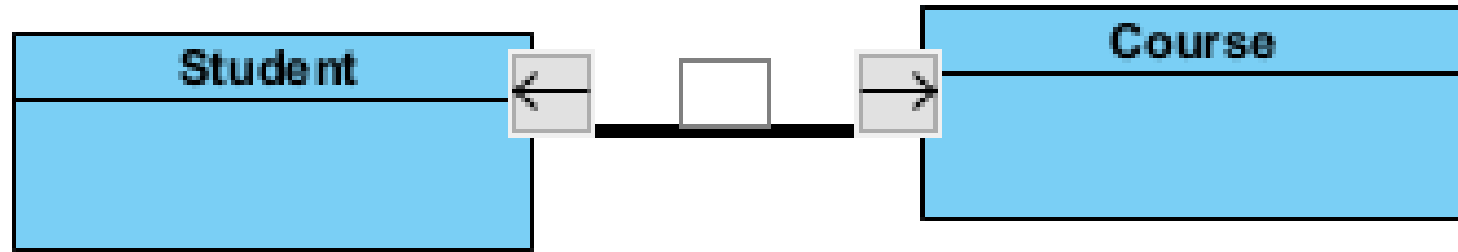
Course

- A. teaches
- B. paints
- ✓ C. enrolls
- D. drinks

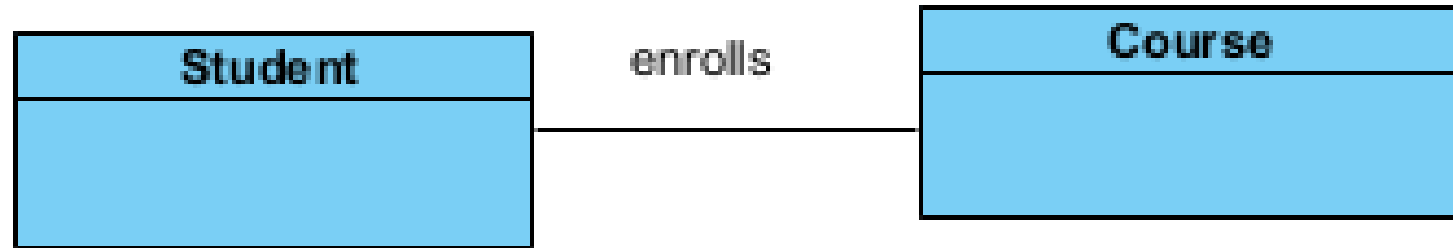
In UML, association names are usually verbs

- For a **general association**, the relationship often represents an **action or interaction**, so we commonly label it with a **verb**.

Association name



Association names should reflect the purpose of the relationship and be a verb phrase.



Name of the association can be shown somewhere near the middle of the association line but not too close to any of the ends of the line.



TOPIC

Topic 4: Association Role Names

Each end of an association is a role, which has various properties, such as:

- name
- multiplicity

A role name identifies an end of an association and ideally describes the role played by objects in the association.

Roles: A role name explains how an object participates in the relationship.



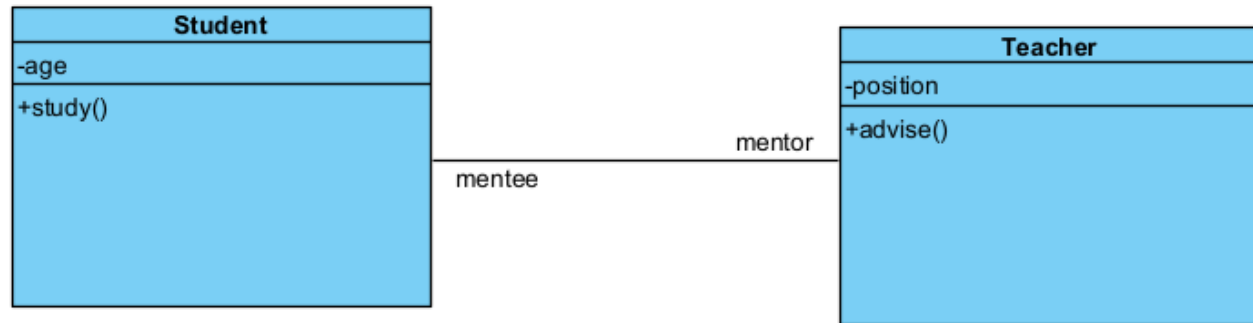
*Association **Wrote** between **Professor** and **Book**
with association ends **author** and **textbook**.*



role name

describes the role of a city in the
Flies-to association

Association



- Association between classes is bi-directional by default.
- A straight line

```
public class Student {

    Teacher mentor;
    private int age;

    public void study() {
        // TODO - implement Student.study
        throw new
        UnsupportedOperationException();
    }

}
```

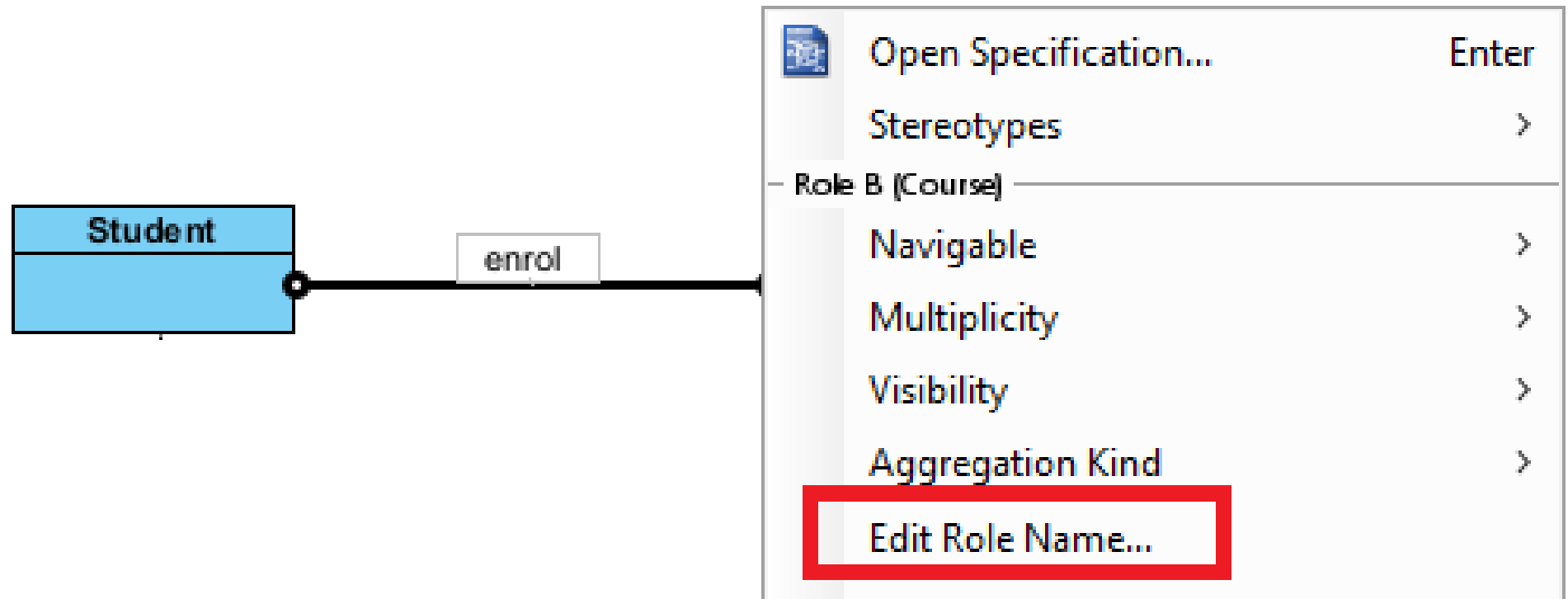
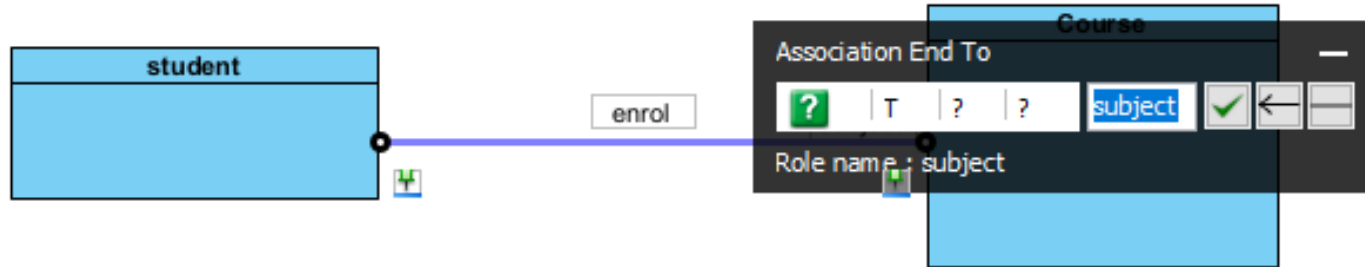
```
public class Teacher {

    Student mentee;
    private int position;

    public void advise() {
        // TODO - implement Teacher.advise
        throw new
        UnsupportedOperationException();
    }

}
```

Visual Paradigm-edit role name





TOPIC

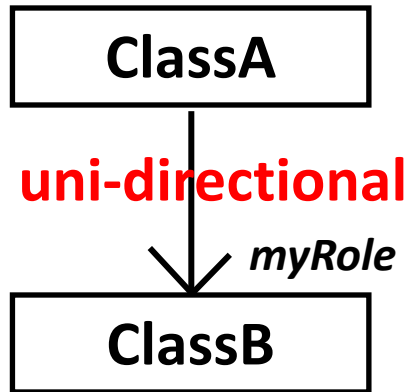
Topic 5: Multiplicity

Multiplicity

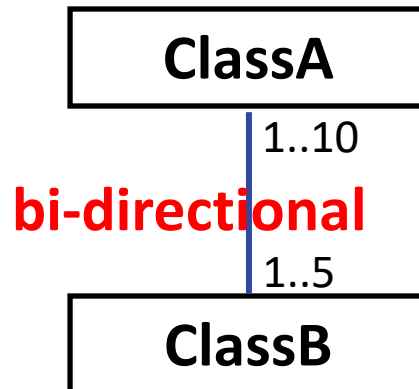
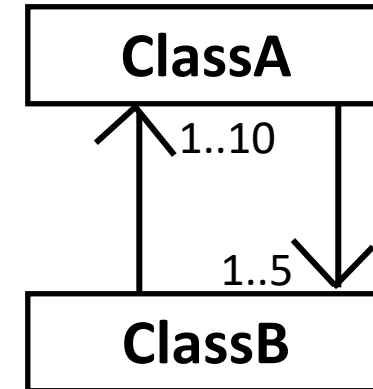
- **Multiplicity** is used to describe how many instances participate in **has-a** relationships, but it doesn't apply to **is-a** (inheritance) or **use-a** (dependency) relationships.
- **Has-a Relationships** (Association, Aggregation, Composition):
 - In these relationships, multiplicity specifies how many instances of one class are related to instances of another class. For example:
 - A **Library** (1) "has" many **Books** (*), or
 - A **Car** (1) "has" 4 **Wheels** (4).

Association in Java Code

Association



```
class ClassA {  
    ClassB myRole ; // attribute  
    .....  
    ....  
}
```



```
class ClassA {  
    ClassB[ ] bObjs = new ClassB[ 5 ]; // attribute  
    .....  
    ..... bObjs[0] = new ClassB(this) ;  
    .....  
}
```

```
class ClassB {  
    ClassA[ ] aObjs = new ClassA[ 10 ]; // attribute  
    .....  
    ..... public ClassB(ClassA a) { aObjs[0] = a ; ...}  
    .....  
}
```

Object
composition

Multiplicities examples:

- 1 Exactly one, no more and no less
- 0..1 Zero or one
- * Many
- 0..* Zero or many
- 1..* One or many

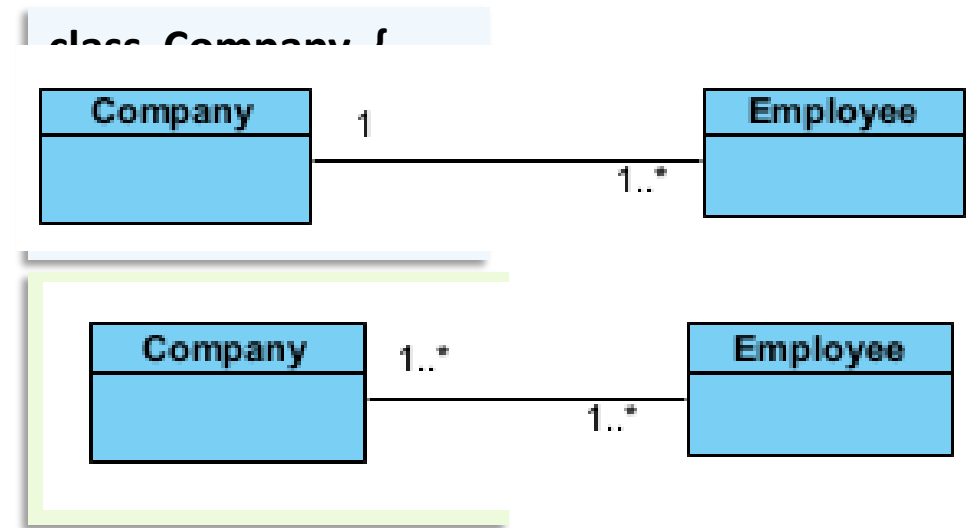
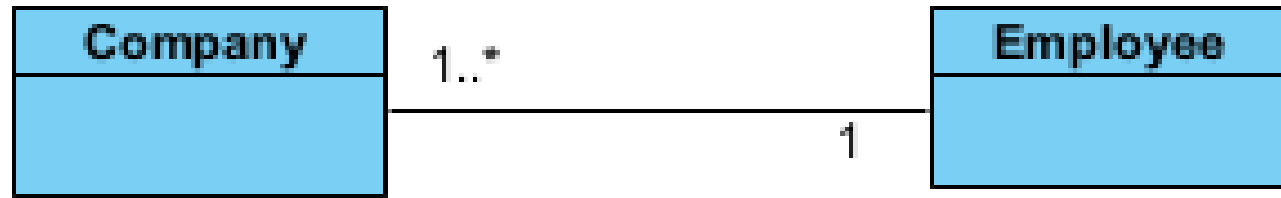
slido



Is the following multiplicity appropriate?

① Start presenting to display the poll results on this slide.

Is the following multiplicity appropriate?

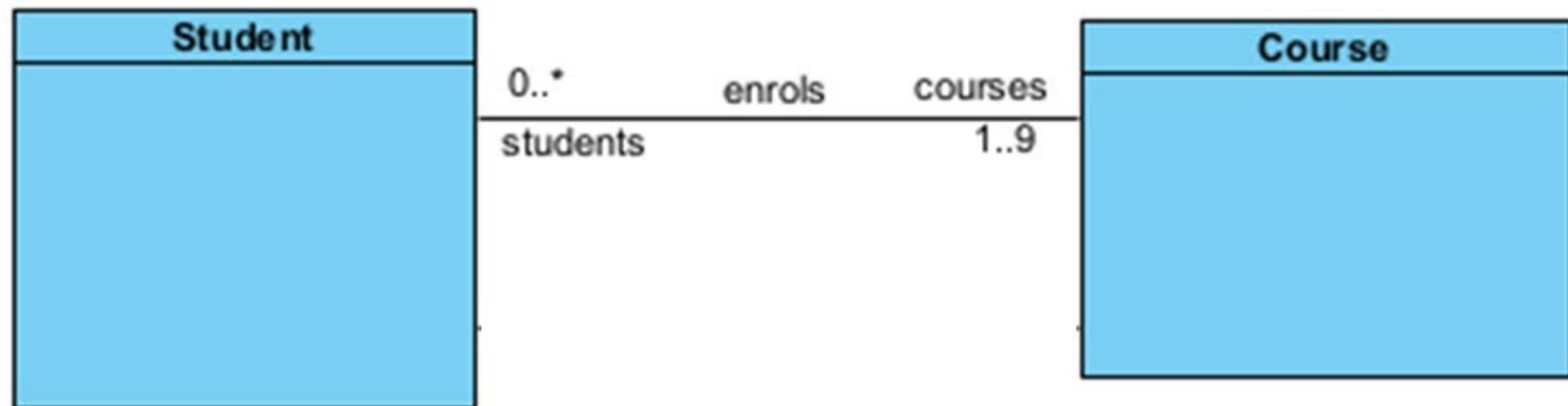


A. Yes

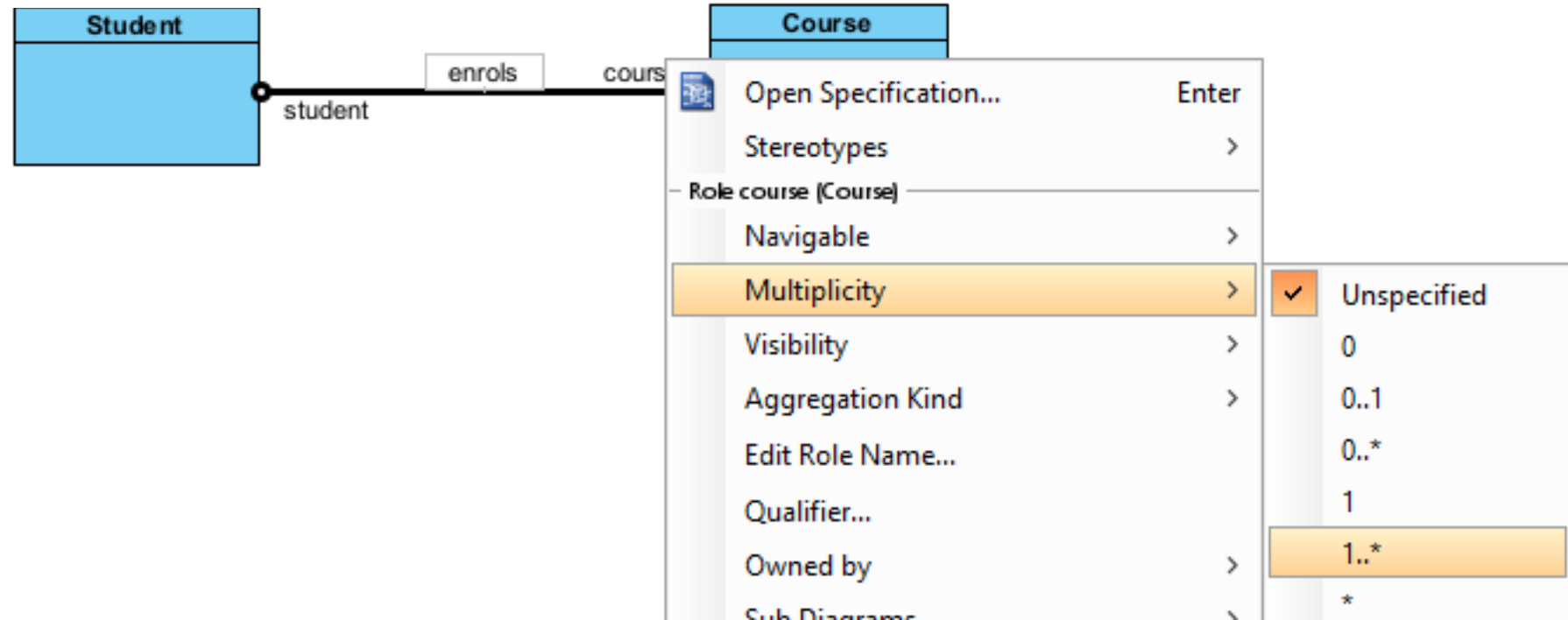
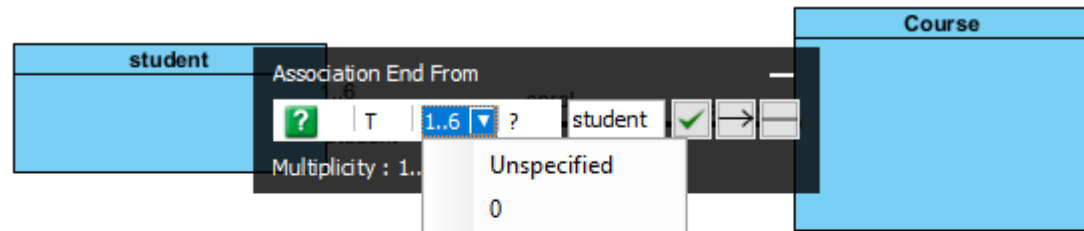
✓ B. No



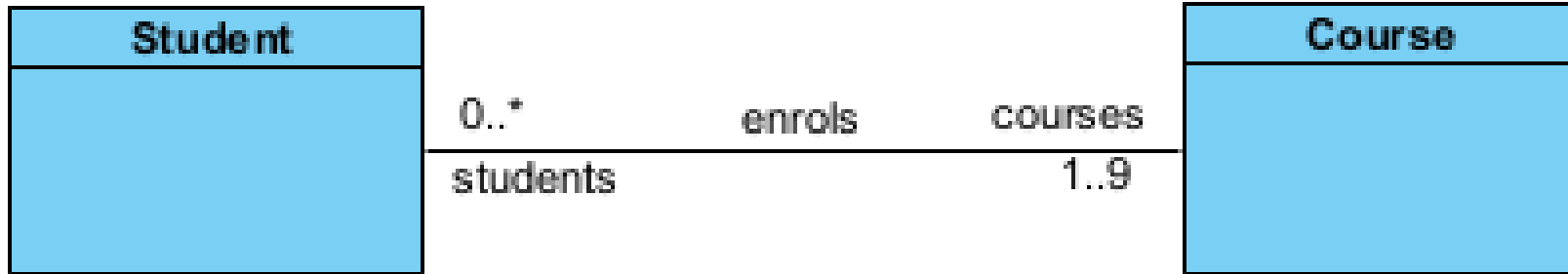
What are the correct statements about the class diagram?



Demo-multiplicity



Demo



```
import java.util.*;

public class Student {

    Collection<Course> courses;

}
```

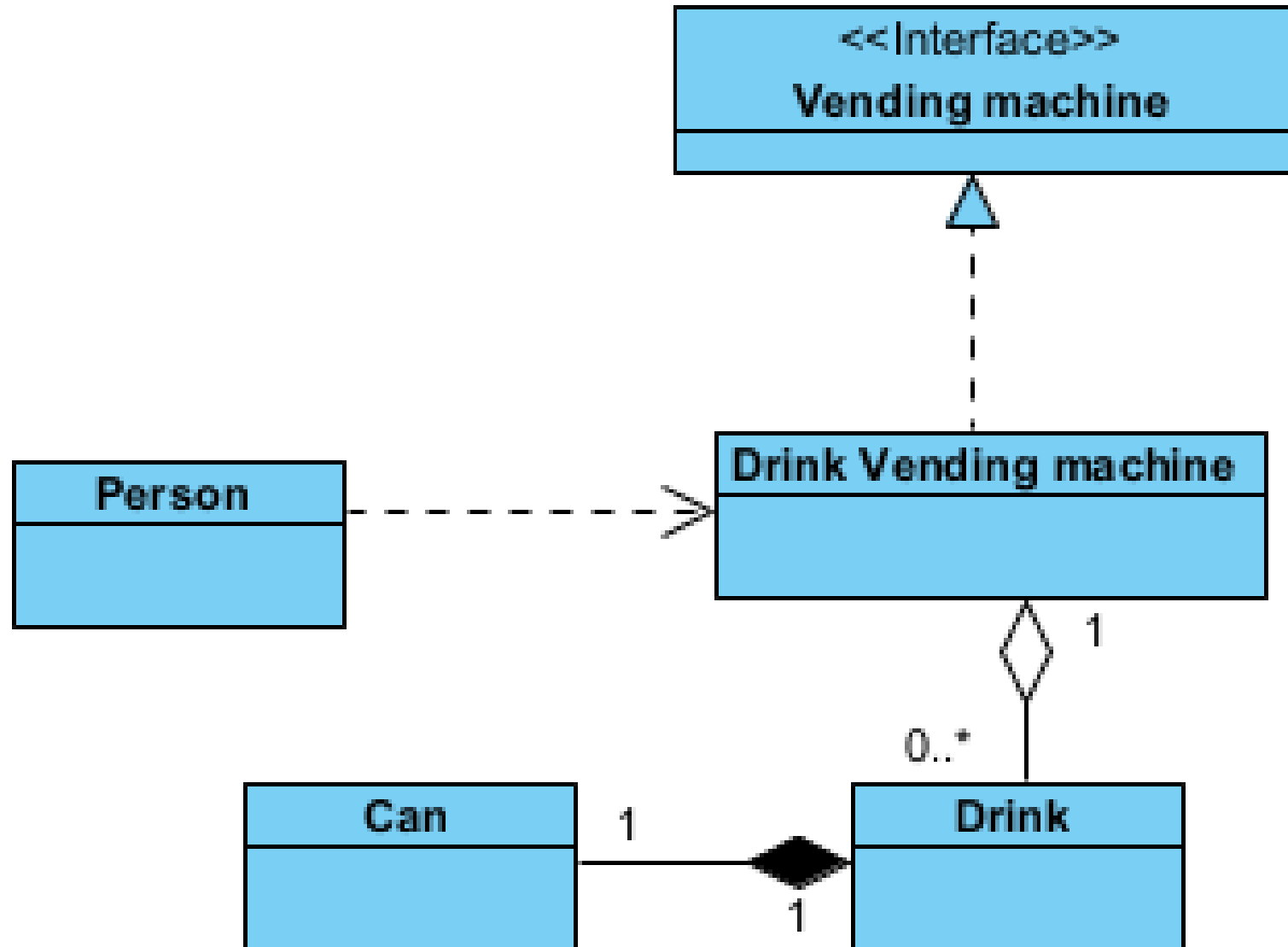
```
import java.util.*;

public class Course {

    Collection<Student> students;

}
```

Multiplication only work on has-a category-association





TOPIC

Topic 6: Constraints

In **UML class diagrams**, constraints are usually written as notes or comments alongside the relationship, specifying conditions like "must", "should", or limits on the relationship.

Constraints typically apply to **has-a** relationships (association, aggregation, composition) to enforce rules about how objects are related. They may also apply to **use-a** relationships (dependency), but they are not typically used in **is-a** (inheritance) relationships.

Constraints in a class-diagram intend to express general statements about the class and its instances.

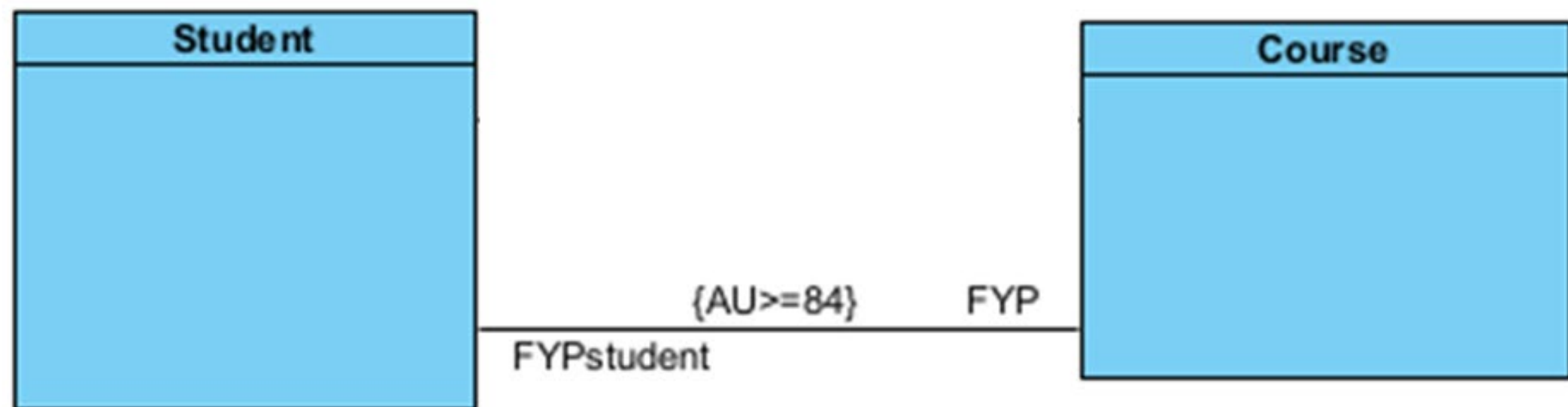
A project which is more than 1 million dollar will require a **project director**.

Maximally, a staff can be a project director for two projects.





What are the correct statements about the class diagram?

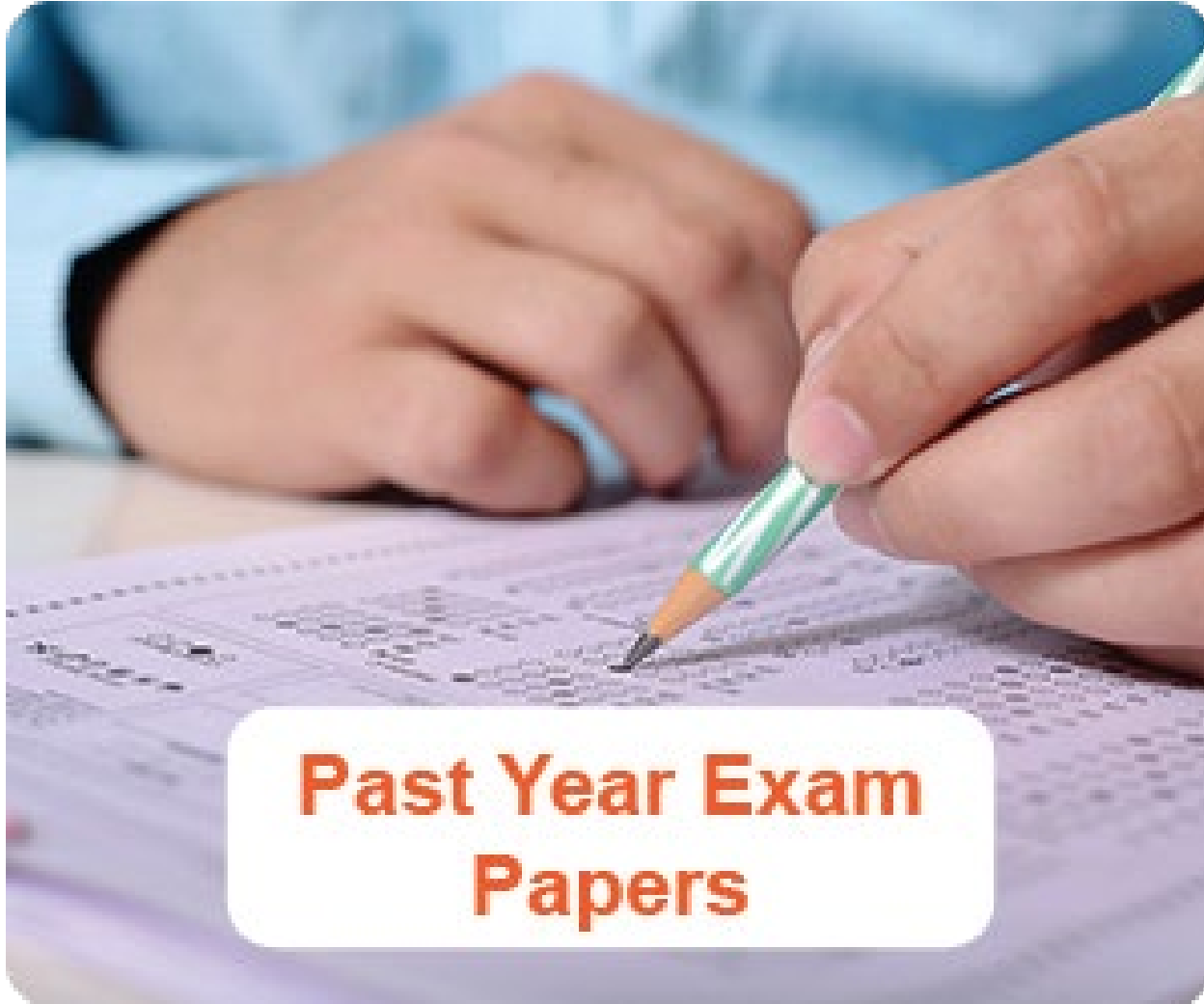


Constraints in Use-a

- **Use-a Relationships (Dependency)**

- Constraints can be applied to **use-a** relationships to define when and how a dependency is invoked.
- **Example:** "A payment processing system must use a payment gateway, but only if the transaction amount exceeds a certain threshold."

PYP revision



**Past Year Exam
Papers**

2020 Nov Q3a

Study the following description of a Work Tracking Application (WTA):

- WTA is a system that maintains a centralized repository of all projects and tasks assigned to staff in an organisation. It contains the project and task details, including the tracking of issues. WTA allows the easy access of relevant information on the staff and projects as well as tracking a project's progress and staff's utilisation rate. A staff can play different roles in projects.
- A project has an identification number, a description, a project manager, the project awarded price and the customer. A project which is more than 1 million dollar will require also a project director. A project is broken down into multiple tasks which are assigned to the same or different staff. Each task will have a project leader, its team members, a task identification, task description and an expected date of completion. Project issues are tracked for every task in the project by assigning an identification number and providing details like title, detailed description, status, reported date and last updated date. A project is funded externally by a customer which the system will store the company name, company registration number, contact person and number.

- WTA also stores the staff details like identification number, name, assigned projects and tasks, skillsets and man-hour rate. Staff will be assigned to ONE to TWO projects but multiple tasks within the same project depending on their skillsets. Skillsets like certification name, the level, awarded date and date of expiry are stored. Every month, each staff is required to clock his/her monthly utilisation details by recording the time spent on the assigned projects and tasks. Details such as the month and year, period of work (from date and to date) and type of work performed. The type of work can be administrative, project, on leave, training and mentoring.

You are tasked to identify the entity classes needed to build the application based on the description above.

Show your design in a Class Diagram. Your Class Diagram should show clearly the relationships between classes, enumeration, relevant attributes (at least TWO), logical multiplicities, meaningful role names, association names and constraint(s), if any. You need not show the class methods.

Steps:

1. Entity classes: Highlight the keywords of classes (Nouns)
 - Those Nouns which have been repeated for a few times
 - Those Nouns which have detailed description/definition
2. Relationship: Highlight the keywords of relationships (Verbs)
 - The verbs indicate “is part of” relationship
 - The verbs indicate “is a special” relationship
 - The verbs indicate normal association relationship
3. Attribute: Those nouns in description/definition of Entity classes
4. Role name: Those nouns which are instances of some entity classes in description/definition of Entity classes
5. Multiplicity: Highlight the words related to quantity
 - a
 - One to two
 - s. e.g., team members

slido



What are the nouns, which have been repeated for a few times in following paragraph?

① Start presenting to display the poll results on this slide.

What are the nouns, which have been repeated for a few times in following paragraph?

WTA is a system that maintains a centralized repository of all projects and tasks assigned to staff in an organisation. It contains the project and task details, including the tracking of issues. WTA allows the easy access of relevant information on the staff and projects as well as tracking a project's progress and staff's utilisation rate. A staff can play different roles in projects.

- A. System, repository, project, staff
- B. Project, staff
-  C. Project, task, staff
- D. WTA, Project, task, staff

slido



What are the nouns, which have detailed description/definition in following paragraph, but not in the previous list?

① Start presenting to display the poll results on this slide.

What are the nouns, which have detailed description/definition in following paragraph, but not in the previous list?

A project has an identification number, a description, a project manager, the project awarded price and the customer. A project which is more than 1 million dollar will require also a project director. A project is broken down into multiple tasks which are assigned to the same or different staff. Each task will have a project leader, its team members, a task identification, task description and an expected date of completion. Project issues are tracked for every task in the project by assigning an identification number and providing details like title, detailed description, status, reported date and last updated date. A project is funded externally by a customer which the system will store the company name, company registration number, contact person and number.

- A. ProjectIssue, customer, description
- ✓ B. ProjectIssue, customer
- C. ProjectIssue, customer, company
- D. Project, ProjectIssue, customer

slido



What are the nouns, which have detailed description/definition in following paragraph, but not in the previous lists?

① Start presenting to display the poll results on this slide.

What are the nouns, which have detailed description/definition in following paragraph, but not in the previous lists?

WTA also stores the staff details like identification number, name, assigned projects and tasks, skillsets and man-hour rate. Staff will be assigned to ONE to TWO projects but multiple tasks within the same project depending on their skillsets. Skillsets like certification name, the level, awarded date and date of expiry are stored. Every month, each staff is required to clock his/her monthly utilisation details by recording the time spent on the assigned projects and tasks. Details such as the month and year, period of work (from date and to date) and type of work performed. The type of work can be administrative, project, on leave, training and mentoring.

- ✓ A. Skillsets, monthlyutilisation, typeofwork
- B. identificationNumber, typeofwork
- C. Skillsets, monthlyutilisation

3. (a) Study the following description of a Work Tracking Application (WTA):

WTA is a system that maintains a centralized repository of all projects and tasks assigned to staff in an organisation. It contains the project and task details, including the tracking of issues. WTA allows the easy access of relevant information on the staff and projects as well as tracking a project's progress and staff's utilisation rate. A staff can play different roles in project

role

constraints

A project has an identification number, a description, a project manager, the project awarded price and the customer. A project which is more than 1 million dollar will require also a project director. A project is broken down into multiple tasks which are assigned to the same or different staff. Each task will have a project leader, its team members, a task identification, task description and an expected date of completion. Project issues are tracked for every task in the project by assigning an identification number and providing details like title, detailed description, status, reported date and last updated date. A project is funded externally by a customer which the system will store the company name, company registration number, contact person and number.

WT³A also stores the staff details like identification number, name, assigned projects and tasks, skillsets and man-hour rate. Staff will be assigned to ONE to TWO¹ projects but

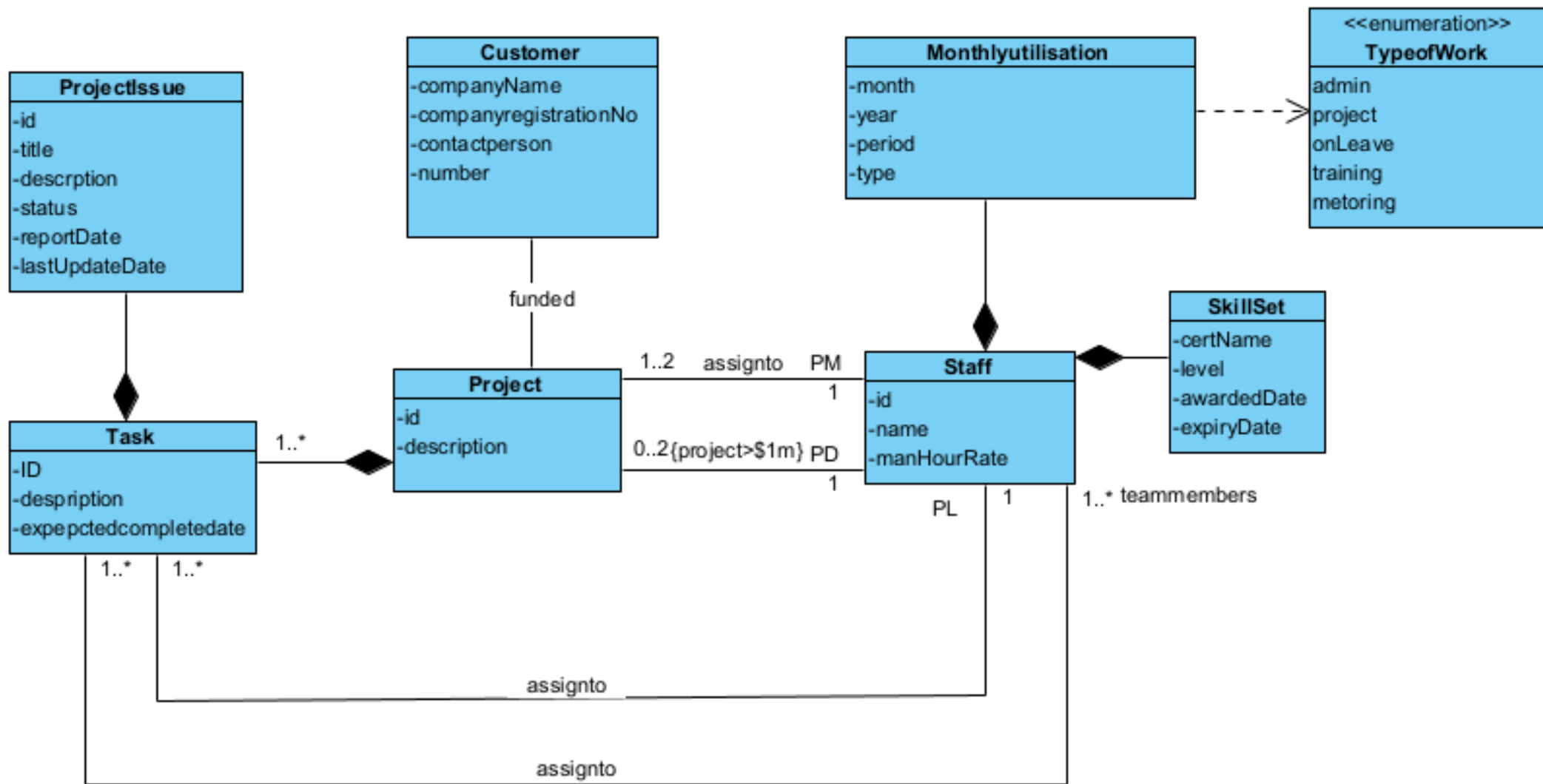
multiple tasks² within the same project depending on their skillsets. Skillsets⁶ like certification name, the level,

awarded date and date of expiry are stored. Every month, each staff³ is required to clock his/her monthly utilisation⁷

details by recording the time spent on the assigned projects and tasks. Details such as the month and year, period of work (from date and to date) and type of work performed.

⁸ The type of work can be administrative, project, on leave, training and mentoring.

enumeration



Correct classes : **5 marks** (esp Staff, Project, Task, Issue, Utilisation)

WorkType as enum : 1 mark

Constraints for project cost > 1 mil to have Proj Dir : 1 mark

Correct use of association, composition : **2 marks**

Attributes : **2 marks** (no private -1)

Correct logical Multiplicity : **2 marks** (0..*/1..*) -- show **0..2**

Association name : **1 mark**

```
public class Project {  
  
    Staff PD;  
    Staff PM;  
    private int id;  
    private int description;  
  
}
```

```
public class Staff {  
  
    private int id;  
    private int name;  
    private int manHourRate;  
  
}
```

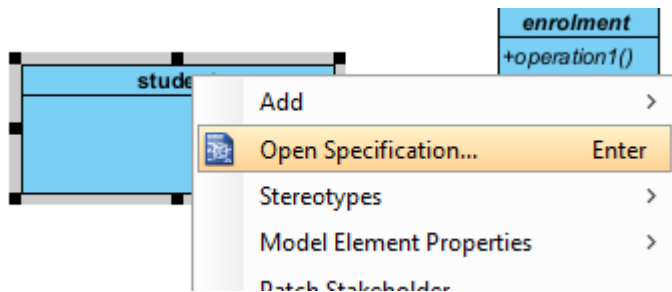
```
import java.util.*;  
  
public class Task {  
  
    Staff PL;  
    Collection<Staff> teammembers;  
    private int ID;  
    private int description;  
    private int expepectedcompletedate;  
  
}
```

```
public class ProjectIssue {  
  
    private int id;  
    private int title;  
    private int description;  
    private int status;  
    private int reportDate;  
    private int lastUpdateDate;  
  
}
```



TOPIC

Topic 7: Abstract class



Class Specification

General Attributes Operatio

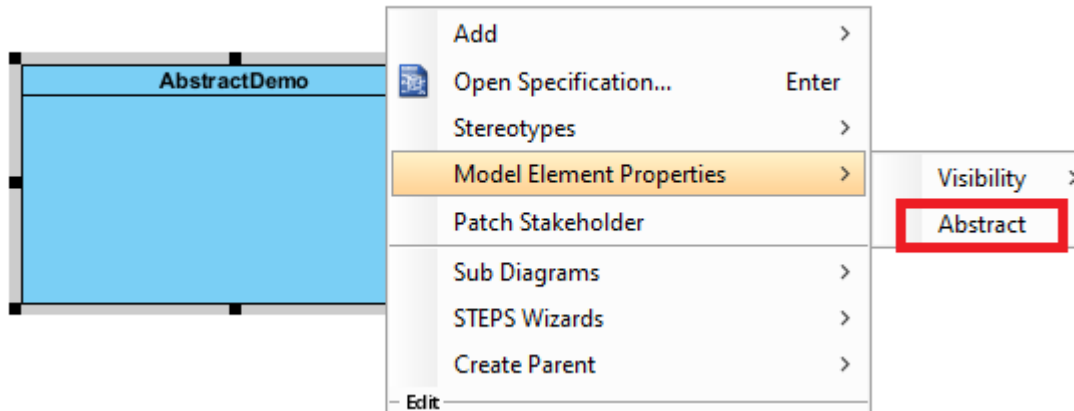
Name:

Parent:

Visibility:

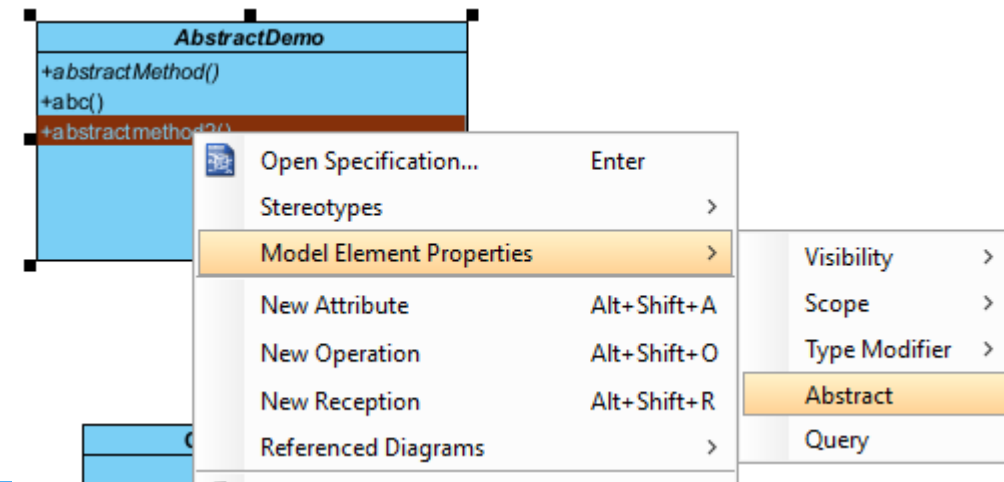
Description:

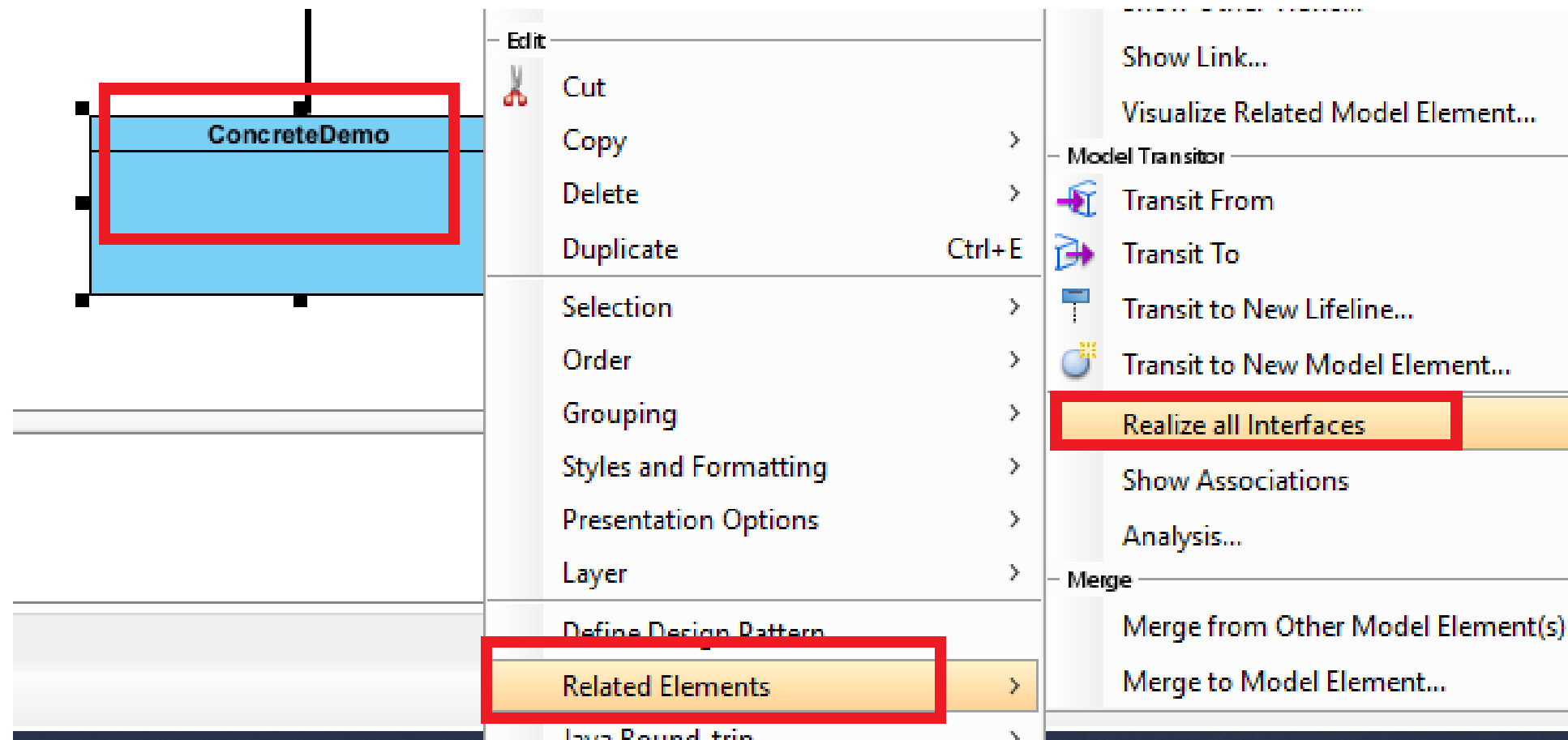
☒ **Abstract**

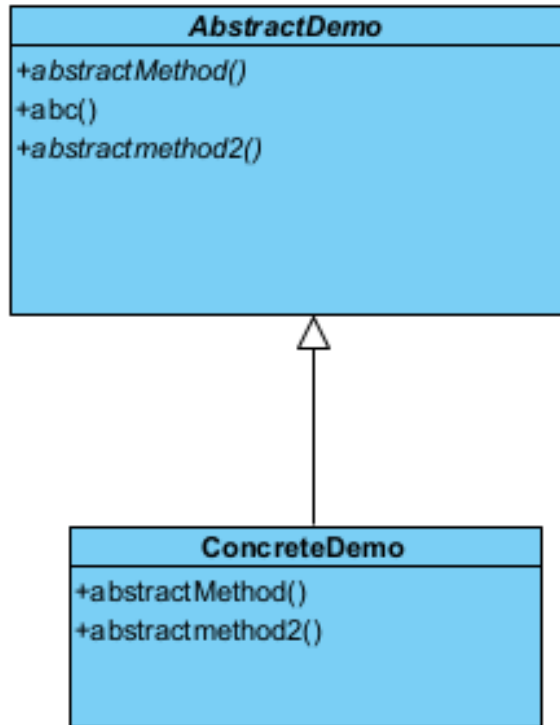


Abstract class

Abstract method







```
public class ConcreteDemo extends AbstractDemo {

    public void abstractMethod() {
        // TODO - implement
        ConcreteDemo.abstractMethod
        throw new
        UnsupportedOperationException();
    }

    public void abstractmethod2() {
        // TODO - implement
        ConcreteDemo.abstractmethod2
        throw new
        UnsupportedOperationException();
    }

}
```

Reference

- Google images